UNIVERSITY
OF HULL

A Low-Level Approach to Lunar Terrain Rendering and Performance Evaluation with Vulkan

being a dissertation submitted in partial fulfilment of the requirements for the degree of

Master of

Computer Science for Games Programming

in the University of Hull

by

Dilara Albayrak

September 2025

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Warren Viant, for his invaluable guidance, support, and encouragement throughout the course of my research. In addition to his supervision, I am deeply thankful for the knowledge and skills he taught during the master's programme, particularly in C++ and related fields, which have been essential to the development of this thesis.

I would like to thank Qingde Li, his Real-Time Graphics course was where I first delved into the rendering pipeline and graphics programming. The insights I gained thanks to his teaching built a foundation for the work presented in this dissertation. Further, I am thankful to him for marking and assessing this thesis.

Abstract

The real-time rendering of large-scale, high-fidelity terrain, such as the lunar surface, presents a significant challenge in computer graphics. High-level game engines often abstract the underlying rendering pipeline, limiting the fine-grained control necessary for advanced performance optimisation. This dissertation details the design, implementation, and performance analysis of a rendering pipeline for a large-scale lunar terrain model, moving beyond high-level tools to gain direct control using the low-level Vulkan graphics API. The primary aim of this study is to implement and assess the performance of advanced rendering techniques for a high-polygon model of the Apollo 11 landing site, derived from NASA's Lunar Reconnaissance Orbiter Camera (LROC) data. The methodology involved several key stages: first, processing raw GeoTIFF data into render-ready assets, including high-resolution meshes and 16-bit heightmaps; second, conducting a preliminary benchmark that established Vulkan's superior performance and stability over DirectX 11, particularly for high-polygon workloads; and third, iteratively developing and testing a series of rendering implementations using Vulkan API. These implementations included a baseline static mesh renderer, a dynamic Level of Detail (LOD) system using hardware tessellation based on camera distance, and both single-pass (multi-view) and multi-pass stereo rendering approaches to simulate a demanding Virtual Reality (VR) workload. The final implementation was also integrated with the OpenXR SDK to deliver a full PC-tethered VR experience. Performance analysis, conducted through systematic, automated camera traversals, yielded several key findings. The hardware tessellation-based LOD system proved highly efficient, rendering visually complex, dynamic geometry with minimal performance cost compared to rendering a dense, static mesh. The comparison of stereo techniques demonstrated that the hardware-accelerated single-pass approach was significantly more performant than the software-driven multi-pass method, highlighting the efficiency gains from modern GPU features. This thesis successfully demonstrates the implementation of a high-performance rendering pipeline for large-scale planetary terrain, providing a quantitative analysis that validates the use of the Vulkan API for demanding scientific visualisation tasks. The findings reaffirm that a dynamic LOD system is a crucial requirement for such applications and provide a practical foundation for future work in real-time simulation and immersive VR experiences.

Contents

Acknowledgements i

Abstract ii

Contents iii

List of Figures v

List of Tables vi

Chapter 1 Introduction 1

1.1 Project Motivation and Scope 1

1.2 Aims and Objectives 2

1.3 Technical Overview 2

1.4 Dissertation Structure 3

Chapter 2 Related Works 4

2.1 Modern Graphics APIs 4

2.2 Real-Time Terrain Rendering 4

2.3 Stereo Rendering and Virtual Reality Systems 5

Chapter 3 Lunar Dataset 7

3.1 Digital Terrain Model (DTM) and GeoTIFF 7

3.2 Data Processing 8

3.2.1 An Overview of Data Processing Code 8

3.2.2 Pixel to Vertex Conversion (2D to 3D mapping) 9

3.2.3 Triangulation and Face Normals 10

3.2.4 Generating Height Maps 11

Chapter 4 Design 12

4.1 Overview of Experimental Rendering 12

4.2 Architectural Design of the Rendering Pipeline 13

4.3 Data Collection Setup 15

Chapter 5 Implementation 16

5.1 Vulkan API Overview 16

5.2 Shaders in Vulkan 18

5.3 Implementation Overview 19
5.4 Mesh (.obj) Rendering 19
5.5 Camera Distance Based LOD with Tessellation Rendering 21
5.6 An Overview of Stereo Rendering 26
5.7 Single-Pass Stereo Rendering 27
5.8 Multi-Pass Stereo Rendering 28
5.9 Multi-Pass Stereo Rendering with Tessellation 30
5.10 Integrating OpenXR into the Vulkan Renderer for VR Viewing 31
5.11 Implementation of Data Collection 34
Chapter 6 Results and Discussion 35
6.1 Mesh Rendering with Varying Resolutions 35
6.2 LOD Rendering with Different Tessellation Levels 36
6.3 Stereo Rendering (Singlepass or Multipass) 38
6.4 Stereo Rendering with and without Tessellation 38
Chapter 7 Conclusion 40
Legal, Ethical, Social and Professional Considerations 42
References 43

List of Figures

Figure 3-1 An illustration of raster data 7
Figure 3-2 A diagram of data processing pipeline 9
Figure 3-3 An illustration of pixel to vertex conversion 10
Figure 3-4 An illustration of triangulation and generating face normal 10
Figure 3-5 Height value histograms 11
Figure 4-1 An overview of the experimental pipeline 13
Figure 4-2 The call graph of the common architecture 14
Figure 4-3 Camera traversal path for data collection 15
Figure 5-1 An overview for simple rendering using Vulkan 17
Figure 5-2 An illustration of geometry detailing by camera distance 21
Figure 5-3 An overview of shaders used for LOD by camera distance 22
Figure 5-4 An illustration of off-axis camera layout for stereo viewing 26
Figure 5-5 Examples of the left eye and the right eye rendering for the same frame 26
Figure 5-6 The call graph of the OpenXR-Vulkan rendering architecture 32
Figure 5-7 An overview of main OpenXR related methods in the implementation 34
Figure 6-1 Visual Results and FPS graphs for rendering of varying mesh resolutions 36
Figure 6-2 Render time histograms for varying mesh resolutions 36
Figure 6-3 Visual Results and FPS graphs for rendering with different max. tessellation levels 37
Figure 6-4 Render time histograms for rendering with different max. tessellation levels 37
Figure 6-5 FPS graphs and render time histograms for stereo rendering 38
Figure 6-6 FPS graphs and render time histograms for multipass stereo rendering 39

List of Tables

Table 5-1 Key differences between implementation variations..... 19

Table 6-1 Initial FPS results for moon terrain rendering 35

Chapter 1 Introduction

The real-time rendering of vast regions remains a significant challenge in the computer graphics field (Müller et al., 2021). This challenge is particularly evident when rendering large terrain models, such as the lunar surface, where high-resolution geometric data is crucial for creating environments built upon real data. Publicly available datasets, such as NASA's Lunar Reconnaissance Orbiter Camera (NASA LROC), enable scientifically accurate and visually enhanced terrain models. This study extends an initial project where GeoTIFF data were used to generate a realistic 3D model of a 1024 m X 1024 m lunar region and deliver it in a virtual reality environment via head-mounted display (Meta Quest 3). A preliminary work conducted within the Unity game engine resulted in a high-polygon mesh of approximately two million triangles paired with 8K resolution textures for the aforementioned lunar region. While high-level tools like Unity game engine offer rapid development, they obscure the underlying rendering pipeline, limiting the fine-grained control necessary for performance optimisation and tailored rendering techniques. This change allows complete low-level control over the rendering pipeline and the exploration of advanced rendering techniques specific to achieving high fidelity.

1.1 Project Motivation and Scope

This project will be a technical development from an academic standpoint. The main goal of this project is to move beyond a high-level engine and achieve direct, low-level control over the graphics pipeline for rendering a larger lunar region, i.e. Apollo 11 Landing Site, a 27.96 km × 4.22 km region. The central focus of this study is to conduct a performance analysis of rendering implementations developed in a low-level graphics API. To this end, initial comparisons between DirectX 11 and Vulkan have been made to determine a more suitable framework for performance-critical applications. Based on initial findings, Vulkan was selected for its cross-platform capabilities and its design, which offers more explicit control over hardware resources. Consequently, while a full VR headset implementation was not pursued, stereo rendering was implemented to simulate the demanding performance characteristics of VR applications, which must efficiently render a scene from two slightly different viewpoints. The project can be divided into three main pillars: 1) developing the rendering pipeline for the Apollo 11 Landing Site, 2) improving the rendering pipeline by adding Level of Detail (LOD) enhancement, and 3) extending the rendering pipeline to stereo rendering. While achieving these, render time and FPS were measured and analysed. The implementation and assessment were performed iteratively. The project's main goal then became an investigation into the performance analysis involved in rendering a high-fidelity terrain in a static scene with moving camera using rendering techniques on the Vulkan Graphics API.

1.2 Aims and Objectives

The main purpose of this dissertation is to implement and assess the performance of the Vulkan graphics API while rendering a high-polygon lunar terrain model across different hardware configurations. This involves moving beyond the abstractions of high-level game engines to gain direct, low-level control over the graphics pipeline using the Vulkan API, through an analysis of various rendering techniques under performance-critical conditions. To achieve this, the following steps were applied:

- To decide the main graphics API to use in this study, a baseline performance comparison between DirectX 11 and Vulkan, for rendering the lunar terrain model, are made and FPS results on two distinct GPUs, an NVIDIA GTX 1050 and an NVIDIA RTX 3070, are assessed.
- To process the raw NASA LROC GeoTIFF data of the Apollo 11 Landing Site into a mesh to be rendered. This involved developing a custom C++ application using the Geospatial Data Abstraction Library (GDAL) to generate both a high-fidelity 3D mesh in .obj format and high-precision 16-bit heightmaps suitable for GPU-based displacement.
- To implement a dynamic LOD system within the more performant Vulkan framework, hardware tessellation approach, based on camera distance to optimise geometric detail, is adopted. This system was designed to offload geometry generation to the GPU, using a simple, low-resolution control grid and the generated heightmap to dynamically create high-fidelity surfaces in real-time.
- To implement stereo rendering within the Vulkan application to analyse the performance impact of rendering the scene twice (left eye and right eye) by simulating a VR-like workload and comparing the performance with mono rendering.
- To validate the renderer's applicability for immersive visualisation, the final implementation was integrated with the OpenXR SDK, enabling a PC-tethered VR experience by streaming the high-fidelity lunar terrain to a head-mounted display.

1.3 Technical Overview

The core of the technical development is centred around the Vulkan graphics API, a modern, low-level interface that provides explicit control over GPU resources and operations. Vulkan was selected after a preliminary benchmark against DirectX 11, where it demonstrated superior performance and stability, particularly when rendering high-polygon workloads, which is a critical requirement for this study. First, raw scientific data from NASA LROC, specifically a high-resolution GeoTIFF Digital Terrain Model (DTM) of the Apollo 11 Landing Site, is converted to a geospatial raster data into render-ready mesh for which a custom data

processing application was developed. This tool leverages the GDAL to parse the 32-bit floating-point elevation data and generate a high-resolution static 3D mesh in the Wavefront .obj format and a high-precision 16-bit heightmap texture. Within the Vulkan framework, several rendering techniques were implemented and evaluated to manage the geometric complexity of the lunar surface. The initial implementation served as a baseline, rendering the static .obj mesh directly. Subsequently, a more advanced LOD was developed using hardware tessellation. This approach replaces the dense static mesh with a simple, low-resolution control grid, offloading the generation of high-fidelity geometry to the GPU, which dynamically subdivides the surface based on camera distance using the 16-bit heightmap for displacement. To simulate the demanding workload of virtual reality applications, stereo rendering was implemented and analysed. Further, to demonstrate the renderer's applicability for immersive visualisation, the pipeline was successfully integrated with the OpenXR SDK, delivering a PC-tethered VR experience.

The developments are made in C++ on Windows 10 using Visual Studio. GLSL (OpenGL Shading Language) is converted to SPIR-V, and it is used to write vertex, pixel, and tessellation shaders, where applicable. Maths libraries or helper libraries (for linear algebra and model loading) are employed where necessary. Additionally, GDAL is used for lunar data processing. A GitHub repository is created and used for version control.

1.4 Dissertation Structure

- **Related Works** delivers a review of the relevant background literature, covering modern graphics APIs, real-time terrain rendering techniques, hardware tessellation, and the performance challenges associated with stereo rendering.
- **Lunar Dataset** presents the NASA LROC (LRO LROC Team, 2020) data used in this study and elaborates on the data processing pipeline developed to convert the raw GeoTIFF format into render-ready assets, namely a 3D mesh and high-precision heightmaps.
- **Chapter 4: Design** outlines the architectural design of the rendering pipelines and the experimental methodology used for the performance analysis and data collection setup.
- **Chapter 5: Implementation** details the main rendering implementation processes, including camera distance-based tessellation approach and stereo rendering.
- **Chapter 6: Results and Discussion** delivers and interprets the quantitative performance data gathered from the experiments, providing an analysis of the different rendering techniques based on metrics such as FPS and render time.
- **Chapter 7: Conclusion** summarises the key findings of the study, discusses their implications, and proposes potential improvements for future work.

Chapter 2 Related Works

This chapter is a review of the academic background associated with the high-fidelity lunar terrain rendering. This review is structured as three pillars. The first one is the architecture of modern graphics Application Programming Interfaces (APIs), which govern the fundamental interaction between the application and the graphics processing unit (GPU). Next subsection briefs real-time terrain rendering, focusing on Level-of-Detail (LOD) techniques designed to manage geometric complexity. Finally, it investigates the performance considerations and optimisation techniques required for stereo rendering and explains the architectural trade-offs between PC-tethered and standalone VR systems.

2.1 Modern Graphics APIs

The choice of graphics API is fundamental for achieving optimal performance and fine-grained control over the rendering pipeline (Shiraef, 2016). DirectX has been a prominent API for Windows-based rendering, while Vulkan delivers cross-platform flexibility. This difference is crucial for deployment: games or engines that run on Linux, mobile, or heterogeneous hardware typically favour Vulkan (Campbell, 2023). DirectX is a relatively high-level, *readily available* graphics API that conceals much of the hardware detail behind driver abstractions. In practice, the DirectX implicitly handles memory management, resource binding, and synchronisation (Microsoft, 2023). By contrast, Vulkan is a modern, low-level API that exposes GPU resources and operations explicitly. In Vulkan, the application must manually allocate GPU memory, record command buffers, and insert synchronization primitives, whereas DirectX would manage those tasks automatically (Overvoorde, 2025). This fundamental design difference means Vulkan gives developers fine-grained control (and responsibility) over GPU execution, whereas DirectX trades some potential performance for ease of use. While DirectX 11 is effectively single threaded, both DirectX 12 and Vulkan are modern, low-level graphics APIs providing explicit control and multi-threading (Shiraef, 2016). Because of its explicit, low-overhead design, Vulkan generally delivers superior CPU-side performance when used effectively. Recent benchmarks show that Vulkan (and DirectX 12) can sustain far more draw calls and utilise multicore CPUs much efficient than DirectX 11 (Yu & Du, 2024; Процик & Коротєєва, 2025).

2.2 Real-Time Terrain Rendering

Rendering large-scale, detailed terrains in real-time poses a challenge in computer graphics (Hedberg, 2022; Lauritsen, 2025). The immense planetary surfaces, such as the Moon, require efficient methodologies to render geometric complexity without overloading hardware (Masood et al., 2022a; Masood et al., 2022b). It is computationally inefficient to render

millions of triangles for all rendering parts in every frame, especially for real-time applications (Lindner et al., 2021). LOD approaches address this challenge, allowing for visual simplifications by adapting the detail of the terrain based on its distance from the camera, i.e. sacrificing the visual quality for farther regions to effectively deliver higher visual fidelity for crucial regions in the rendering view (Hoppe, 1998; Luebke, 2003; Hosny et al., 2020; Hedberg, 2022). Two notable LOD approaches for terrain rendering are quadtree-based solutions and tessellation-based solutions (Etienne, 2023; Wendt, 2023). A study conducted by Wendt (2023) directly compares quadtree and tessellation LOD solutions for planetary terrain rendering on modern hardware. While quadtree methods showed overall better performance and control, tessellation produced a richer topology and could automatically add detail. Quadtree solutions typically build the terrain mesh on the CPU, with higher detail closer to the viewer. They offer good performance and greater control over detail levels. However, this can shift a considerable processing load to the CPU (Masood et al., 2022b; Wendt, 2023). In contrast, tessellation-based methods leverage the GPU to subdivide areas of a basic low-resolution model, adding detail to areas close to the viewer. This approach assigns detail generation to the GPU, which can be favourably efficient (Cantlay, 2011; Masood et al., 2022b; Wendt, 2023).

2.3 Stereo Rendering and Virtual Reality Systems

Stereo rendering involves rendering the scene from two slightly different viewpoints, a process that inherently increases the computational workload. This technique is the foundation for creating the illusion of depth and facilitating 3D viewing (Blondé et al., 2010). There are two major approaches for stereo rendering: multi-pass and single pass (multi-view). Multi-pass rendering is a simpler approach, rendering the scene twice, once for the left eye and once for the right eye, each with its own view/projection matrix. With single-pass stereo rendering, the GPU can execute the draw commands once and output to two (or more) viewports/layers simultaneously, differing only by view-specific parameters like the view matrix (Krupip, 2019; Chaurasia et al., 2020). The performance impact of this stereo rendering process, compared to mono rendering, is evaluated in this study's performance analysis. Optimisations for dual-eye rendering are critical in VR, such as Instanced Stereo and Multiview settings, which enable parallel rendering for both eyes, reducing the computational burden compared to rendering each eye separately (Hogan et al., 2022). Maintaining a high and stable frame rate, typically around 60-72 frames per second (FPS) or higher, is essential for a smooth and pleasant user experience in VR (Lai et al., 2017; Jansen et al., 2023) helping to minimise motion sickness and other undesirable effects that degrade user experience (Suwandi et al., 2024). Stereo rendering is still fundamentally more demanding as it requires efficient GPU utilisation and careful management of CPU load, as the CPU can become a bottleneck due to tasks like draw call preparation, physics simulation, and lighting (Wong, 2016). Hence, optimizations like

multithreaded rendering, careful use of GPU culling, and foveated rendering are often employed (Wang et al., 2023). In summary, stereo rendering combines API-level features (single pass or multi pass rendering), runtime frame synchronization and prediction, and specialized hardware (high-refresh panels) to meet the low-latency, high frame rate demands of immersive VR.

The earlier VR systems, such as the Oculus Rift and HTC Vive, adopted a PC-tethered (PC VR) architecture, where the HMD is connected to a powerful personal computer via a physical cable. In this system, all computationally intensive rendering and simulation tasks are loaded to the PC's high-end CPU and GPU (Lai et al., 2017; Rendevski et al., 2022). This approach allows for significantly higher visual fidelity, enabling virtual environments with large polygon counts, complex dynamic lighting and shadows, and advanced post-processing effects. The rendering pipeline for PC VR is specifically designed to prioritise low and stable latency over maximum hardware utilisation, which often results in the CPU becoming a performance bottleneck that requires careful optimisation, as the GPU must frequently wait for the CPU to prepare rendering jobs (Wong, 2016). While this architecture excels in graphical power, the physical tether has been identified in the literature as a significant drawback, as it restricts the user's freedom of movement, can disrupt the sense of immersion, and poses a potential safety hazard (Lai et al., 2017; Rendevski et al., 2022). In contrast, standalone VR systems prioritise mobility and accessibility by integrating all necessary components directly into the HMD itself (Rendevski et al., 2022; Sukaridhoto et al., 2023). This all-in-one design eliminates the need for any external computer or cables, affording the user complete freedom of movement (Rendevski et al., 2022). However, this mobility comes at the cost of computational power. Standalone headsets rely on mobile processors (e.g., Qualcomm Snapdragon XR2) and integrated GPUs, which are considerably less powerful than their desktop counterparts due to strict constraints on power consumption and heat dissipation (Sukaridhoto et al., 2023). Consequently, developers creating applications for these devices must employ heavy performance optimisation to achieve the required 72 FPS target (Hosny et al., 2020; Al-Jundi & Tanbour, 2022). As Hosny et al. (2020) demonstrated, a high-quality application that runs smoothly on a PC VR system would be unusable on standalone hardware without extensive optimisation, suffering from debilitatingly low frame rates (Hosny et al., 2020). Common optimisation strategies include reducing the number of polygons, simplifying lighting and textures, and using techniques such as static batching and GPU instancing to minimise draw calls (Lai et al., 2017; Rendevski et al., 2022; Sukaridhoto et al., 2023). The need for such distinct optimisation strategies stems directly from this architectural divide. Ultimately, the choice between these platforms represents a fundamental trade-off between hardware limitations and user experience.

Chapter 3 Lunar Dataset

This chapter presents the data resource for the technical developments performed in this thesis. These developments heavily rely on surface elevation data provided by NASA LROC (LRO LROC Team, 2020). This raster-based representation is parsed to obtain precise height values, enabling smooth rendering. A custom C++ implementation is built to parse the data, facilitating 3D mesh models and normalised height maps. The elevation data and data processing are explained in this chapter.

3.1 Digital Terrain Model (DTM) and GeoTIFF

A Digital Elevation Model (DEM) is a Digital Terrain Model (DTM) that delivers the height information of a terrain surface. It has extended versions such as Digital Surface Maps (DSM). This study uses DTM to obtain height information of the moon's surface. For the focus of this thesis, i.e. the rendering pipeline, the DTM of the Apollo 11 Landing Site by NASA LROC, in a GeoTIFF format, has been used. GeoTIFF is a variation of the raster image file format that stores pixel-based data and relevant geo-referenced information to create a file of geospatial raster data. A raster image illustration is shown in the Figure 3-1. A single-band GeoTIFF file contains a single value per pixel and is interpreted as a grayscale image. Other versions of a single-band GeoTIFF include palette images, which use a defined set of colours and values to display categorized raster images and elevation data where each pixel value is interpreted as a vertical elevation to build a 3D terrain model. Additionally, GeoTIFF files can hold multiple bands of data, multiple values per pixel to be used together in the analysis and display of the region of interest (Ritter & Ruth, 1997; Lemenkova & Debeir, 2023).

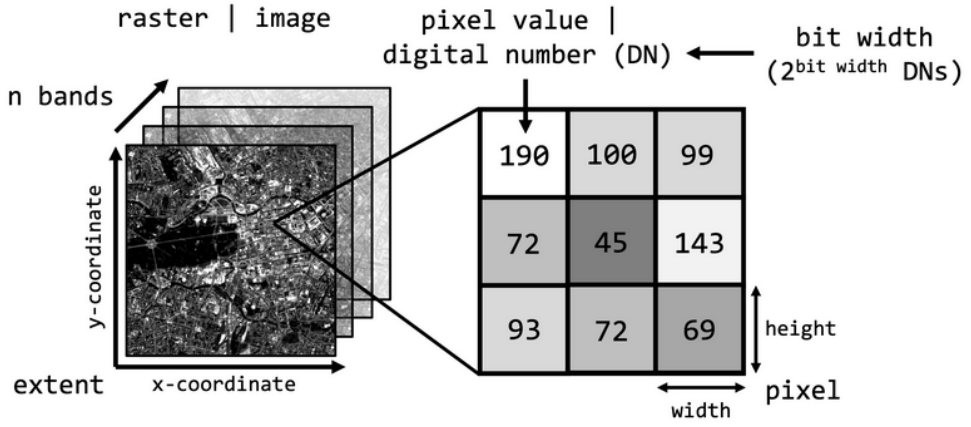


Figure 3-1 An illustration of raster data
Source: Department of Geography, Humboldt-Universität zu Berlin, 2023 https://eol.pages.cms.hu-berlin.de/geo_rs/S03_image-prop2.html#Raster_stacking

The dataset used in this study is a single-band, 32-bit floating-point raster, where each pixel keeps an elevation value in metres relative to a lunar reference sphere. The data is stored in one band, and the “NoDATA” value is defined as -3.40282266e+038. The dataset was delivered by the LROC Science Operations Center at Arizona State University and the GDAL geospatial data library. Data acquisition occurred during the LRO Science Mission phase on 22 January 2011, between 18:55:49 and 20:49:11 UTC, and the final DTM product was generated on 10 October 2020. The projection is equirectangular in a planetocentric coordinate system, with a positive-east longitude convention. The dataset is centred at 1.0° N latitude and 180.0° E longitude, covering a spatial distance from 0.314609° to 1.2365388° N latitude and 23.3722758° to 23.5115296° E longitude. The resolution is 2 metres per pixel, corresponding to a map resolution of 15,161.675 pixels per degree. The raster grid consists of 13,978 lines (rows) and 2,111 samples (columns), yielding a total coverage of approximately 27.96 km × 4.22 km region. This resolution and coverage make the dataset suitable for high-fidelity 3D surface reconstruction and scientific analysis, particularly in the context of lunar landing site studies (LRO LROC Team, 2020).

3.2 Data Processing

The data is processed to create an input for the rendering pipeline. The data processing is implemented in C++ using the GDAL library for raster data handling, with additional custom algorithms for geometry generation, normal calculation, and image encoding. Two major outputs are generated: a high-fidelity 3D mesh in Wavefront OBJ format, and heightmaps in both 8-bit (0-255 range) and 16-bit (0-65,535 range) PNG formats. 16-bit PNG is generated to accurately keep height values parsed from the initial data source, the 32-bit GeoTIFF. The data processing is performed at full spatial resolution by default, with optional downscaling parameters to reduce the output size when required. All outputs are spatially normalised to a centred coordinate system to maintain consistent alignment across mesh and texture assets.

3.2.1 An Overview of Data Processing Code

The mesh generation module (`MeshGenerator::generateMesh`) begins by reading elevation values from the GeoTIFF raster on a scanline-by-scanline basis, respecting the *NoDATA* value to exclude invalid samples. The dataset’s `geoTransform` parameters are used to project pixel indices into real-world cartesian coordinates, ensuring geometric accuracy in metres relative to the lunar reference sphere. Vertices are stored in contiguous memory (vector), and a vertex map is maintained to track spatial-to-index correspondence for subsequent face construction. Once all vertices are loaded, the model is re-centred to the origin by calculating its axis-aligned bounding box and translating each vertex such that the geometric centre lies at (0,0,0). Surface normals are computed per vertex by aggregating the normals of adjacent triangular faces. This

process is parallelised using OpenMP, allowing efficient normal calculation even for multi-million-vertex datasets. Normal values are then normalised to unit length to ensure consistent shading behaviour. The mesh is exported in the Wavefront OBJ format, with explicit vertex (`v`), normal (`vn`), and face (`f`) definitions. Each triangular face includes normal indices for per-vertex shading accuracy. Figure 3-2 shows a UML diagram of the data processing pipeline that delivers both the mesh and the heightmaps .OBJ mesh and heightmaps. The code resources are available in the GitHub repository (Albayrak, 2025a).

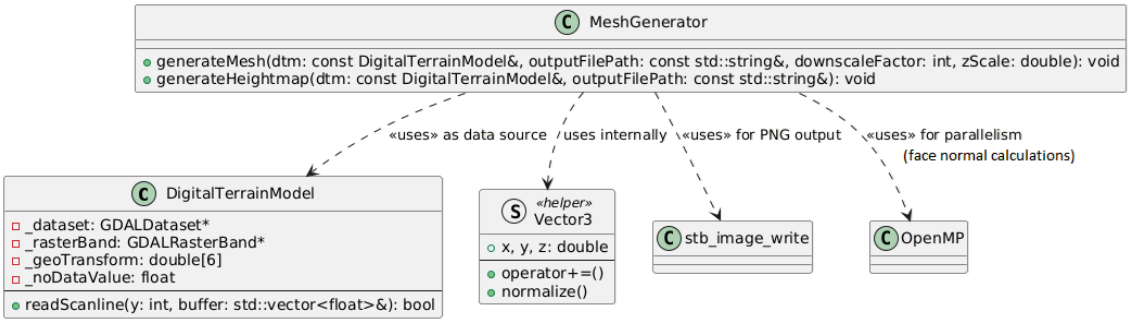


Figure 3-2 A diagram of data processing pipeline

3.2.2 Pixel to Vertex Conversion (2D to 3D mapping)

A GeoTIFF DTM stores elevation as a 2D raster grid, rows and columns of pixels, where each pixel's value represents height at a certain geographic location. For rendering or mesh generation, we must transform each pixel's **(X, Y)** image coordinates into **(x, y, z)** 3D world coordinates: X and Y are flat positions in projected space, whereas Z is the elevation value (metres above/below reference surface). The mapping from pixel indices to projected coordinates involve rotations, offsets, and coordinate origin shifts, illustrated in Figure 3-3. That's where we use GDAL's `geoTransform` array, keeping six elements:

1. Top left X (`geoTransform[0]`): The projected X coordinate (metres or degrees) of the centre of the upper-left pixel. This sets the horizontal origin in real-world space.
2. Pixel width (`geoTransform[1]`): The size of one pixel in the X direction, in the same units as `geoTransform[0]` (e.g. metres/pixel). Positive for left-to-right pixel order.
3. Rotation term X (`geoTransform[2]`): How much the X coordinate changes per pixel step in the Y direction. Non-zero if the raster is rotated relative to the projection grid. Zero means perfectly aligned east-west scanlines.
4. Top left Y (`geoTransform[3]`): The projected Y coordinate (metres or degrees) of the centre of the upper-left pixel. This sets the vertical origin in real-world space.
5. Rotation term Y (`geoTransform[4]`): How much the Y coordinate changes per pixel step in the X direction. Non-zero if the raster is rotated. Zero means north-up without skew.

6. Pixel height (`geoTransform[5]`): The size of a pixel in the Y direction. For north-up, this is negative because Y decreases as you go down image rows (in raster coordinates, row index increases downwards, but in projected coordinates, north is up).

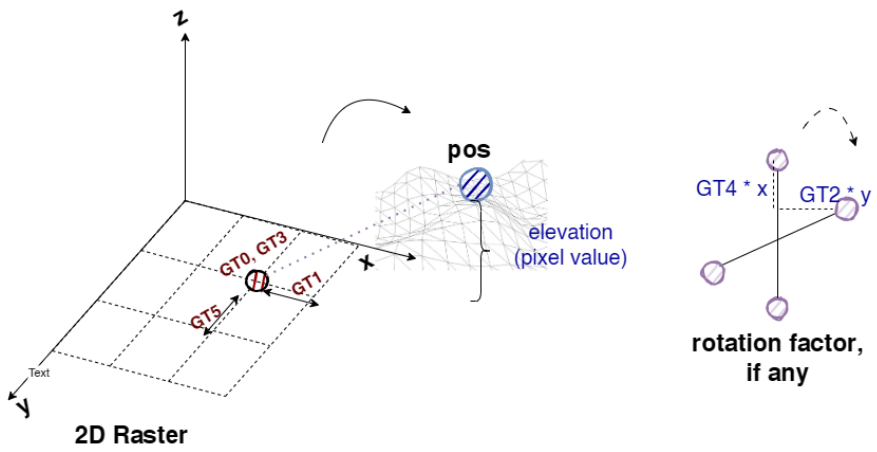


Figure 3-3 An illustration of pixel to vertex conversion

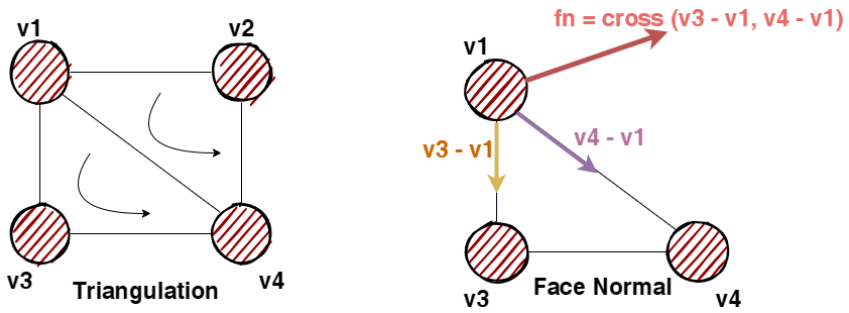


Figure 3-4 An illustration of triangulation and generating face normal

3.2.3 Triangulation and Face Normals

The raster data structure supports a regular grid layout, where each pixel (or elevation sample) corresponds to a vertex in the mesh. Triangulation is required to convert this grid of vertices into a set of planar faces that can be rendered by standard graphics pipelines. In this implementation, four adjacent vertices are grouped into a quad, which is then subdivided into two triangles. The subdivision pattern must remain consistent across the entire grid to ensure a coherent mesh surface without cracks or misaligned edges. For each quad, the diagonals are chosen such that the triangulation order follows a predictable and uniform convention, typically from the top-left to the bottom-right vertex. This consistency is critical for normal computation and prevents shading artefacts. Once the mesh faces are established, face normals are calculated for shading purposes. The normal vector of a face is a unit vector perpendicular to the surface, used by rendering engines to determine light reflection and shading intensity. For a triangle defined by three vertices v_1, v_2, and v_3, two edge vectors are first computed by subtracting vertex positions, and the cross product of these edge vectors

outputs the face normals. Figure 3-4 An illustration of triangulation and generating face normal briefly shows triangulation and face normal generation.

3.2.4 Generating Height Maps

The heightmap generation (`MeshGenerator::generateHeightmap`) creates raster representations of the terrain's vertical displacement (along the Z axis). These are derived from the same DTM source data used in mesh generation, with identical centring and scaling operations to ensure spatial consistency between the mesh and heightmap outputs. First, the vertical range of the processed elevations is determined, and a centre-z offset is calculated to align the midpoint of the elevation distribution to zero. This offset is subtracted from all displacement values to balance positive and negative height variations. *The elevation values are scaled by 1.5 by for better and more distinctive visualisations.* The final displacement Z-axis range used in the rendering pipeline is (-196.42 to 196.42).

- For 8-bit heightmaps, the adjusted displacements are linearly normalised to the range [0, 255] and stored in a single-channel PNG.
- For 16-bit heightmaps, the displacements are mapped to the full [0, 65535] range, greatly increasing displacement precision. This is particularly important for tessellation workflows, where fine-scale elevation detail must be preserved to avoid artefacts in dynamically subdivided meshes.

Both heightmap formats fully exploit the original dataset's dynamic range and preserve the terrain's fine structural details. As a validation, we visualised the original data and the generated height map. Figure 3-5 Height value histograms shows the height value distributions of the 32-bit GeoTIFF and the 16-bit height map. Note that GeoTIFF represents data with floating-point numbers, whereas the height map (right histogram) stores elevation data as integers, so their ranges are different. By matching the geometric transformations applied to the mesh, these textures can be directly used for displacement mapping without requiring further alignment or rescaling.

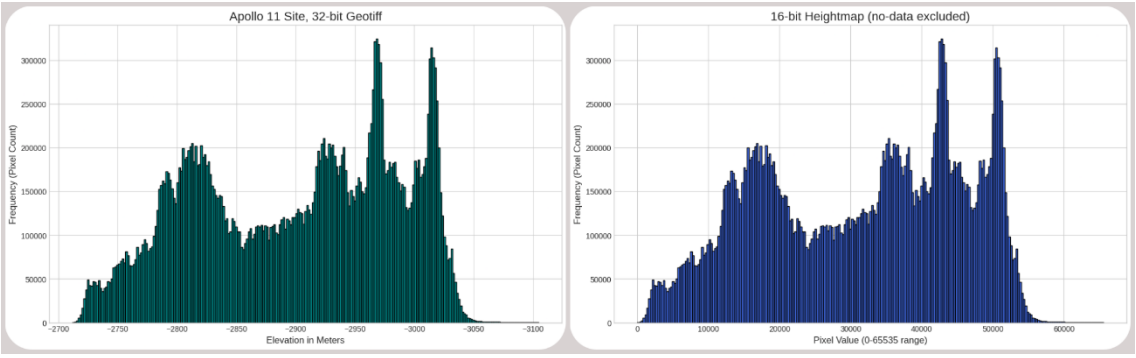


Figure 3-5 Height value histograms
Left is the original GeoTIFF data and the right is the 16-bit heightmap, num. of bins 256

Chapter 4 Design

This chapter explains the work design conducted for this dissertation and illustrates the architecture of the Vulkan implementations. The initial consideration of this design was the methodical evaluation and selection of the core graphics API as this choice forms the foundation for all subsequent performance-critical development. At the early stages of the study, the graphics API best suited to the project's requirements were analysed. To this end, varying resolutions of a 1 km X 1 km moon region (from the Apollo 11 landing site) were rendered using Vulkan or DirectX 11. This initial benchmark was designed to assess each API's ability to handle high-polygon workloads and to measure their raw draw call throughput under controlled conditions. Our purpose was to select one of these APIs and refine the rendering pipeline in line with academic research and performance optimisation strategies.

In Results and Discussion chapter, the initial findings for two separate hardware (NVIDIA GTX 1050 and NVIDIA RTX 3070) are given (see Table 6-1). These GPUs were specifically selected to represent two distinct tiers of hardware capability: a mainstream, older-generation card and a high-performance, modern-generation card, allowing for a broader analysis of API performance across different hardware profiles. The results reveal that Vulkan performs better, and DirectX fails to render as the resolution increases. Considering the size of the rendered terrain would be expanded to the entire Apollo 11 site (27.96 km × 4.22 km as stated in section 3.1), DirectX was not eliminated because of its lower performance, but these failed rendering scenario (i.e. yielding bad allocation). For a project focused on rendering such a large-scale dataset, stability and the capability to scale to high geometric complexity were non-negotiable requirements, making these rendering failures a critical deciding factor. These results also demonstrate that obtaining results on a less powerful device is more meaningful for subsequent rendering variations, as powerful hardware such as the NVIDIA RTX 3070 may mask the effects of rendering optimisation (e.g. LOD design) and differing rendering workloads (mono rendering compared to stereo rendering). Then, results and comparisons in Chapter 6 are obtained by running the implementations on NVIDIA GTX 1050.

4.1 Overview of Experimental Rendering

After real-world moon GeoTIFF data is processed as explained in section 3.2, the generated .obj mesh is rendered using the Vulkan API for 3 resolution levels, low level, medium level, and high level with 22k triangles, ~1.1m triangles and ~6.3m triangles, respectively. This initial stage served as a baseline to establish the raw performance characteristics of rendering static geometry before introducing more complex, dynamic systems. FPS and render time comparisons for these initial renderings are shown in the Results

and Discussion chapter. Then, the heightmap obtained by the data processing implementation (as explained in section 3.2) is used, and tessellation by camera distance LOD is applied. The objective here was to transition from a static mesh to a dynamic geometry solution, leveraging the GPU's hardware tessellation units to adapt terrain complexity in real-time. 3 levels of tessellation applied, where the maximum tessellation level is 1 for the lowest quality (meaning, no tessellation), 8 for a higher quality and 16 for the highest quality. These levels directly controlled the subdivision factor, allowing for a granular analysis of the trade-off between visual fidelity and GPU workload. For stereo rendering, two different approaches are adopted: single-pass and multi-pass. Finally, tessellation as an LOD optimisation is applied for multi-pass stereo rendering. This final variation represents the most complex rendering scenario, combining the GPU-intensive workload of dynamic tessellation with the demanding requirements of stereo rendering to test the limits of the optimized pipeline. FPS and render time results for these rendering configurations are also discussed in the Results and Discussion chapter. Figure 4-1 briefly illustrates these various implementation steps.

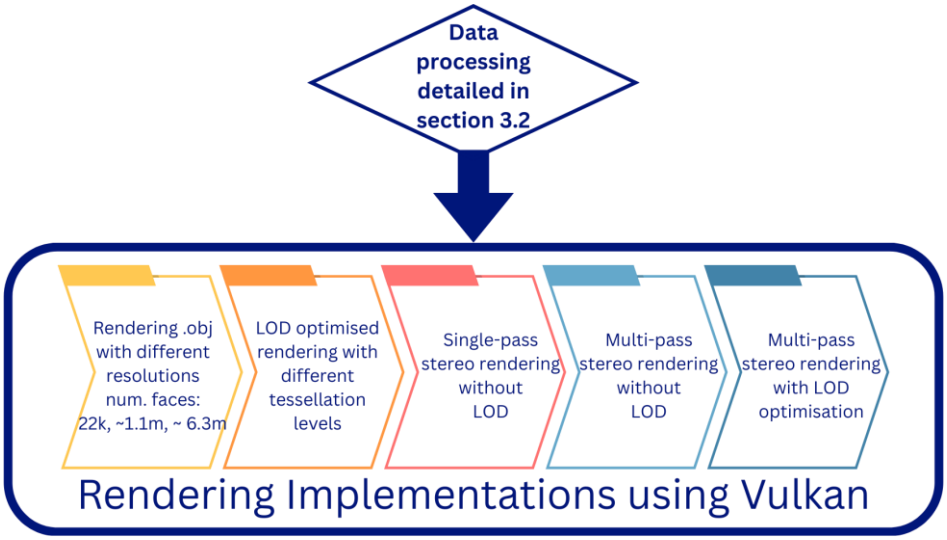


Figure 4-1 An overview of the experimental pipeline

4.2 Architectural Design of the Rendering Pipeline

Across all five rendering implementations, a common architectural foundation was established using a single `TerrainRenderer` class to manage the application lifecycle. A common call graph shared among different rendering configurations is given in. This foundational structure shows the flow through four primary stages: `initWindow()`, `initVulkan()`, `mainLoop()`, and `cleanup()`. While the high-level sequence of Vulkan object creation within `initVulkan()` remains similar, the specific configurations within these steps and the rendering paradigm employed in the `mainLoop()` may diverge to meet the goals of each implementation stage.

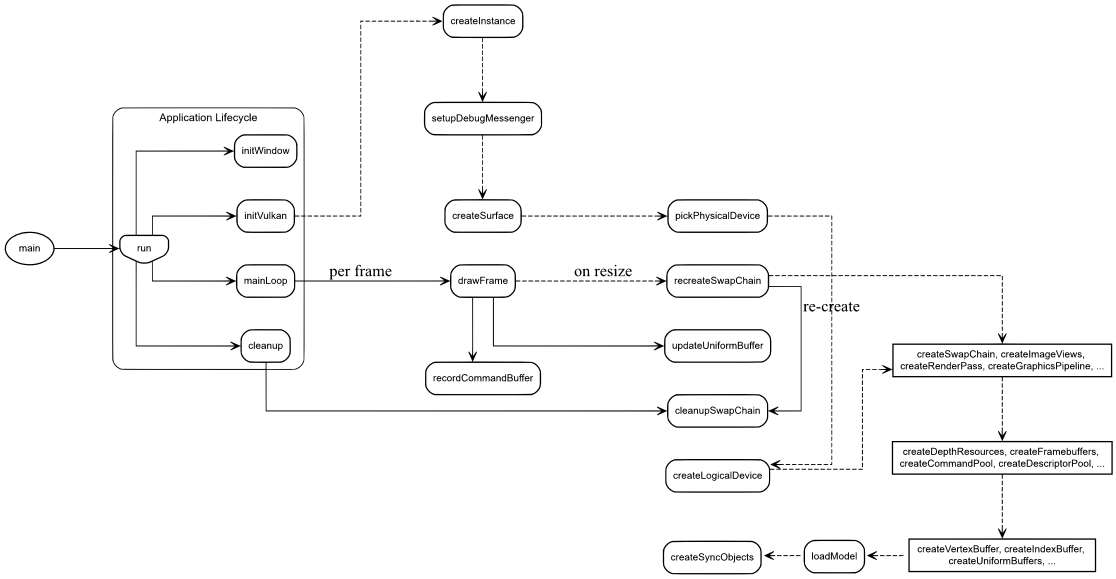


Figure 4-2 The call graph of the common architecture

The initial scenario, a high-resolution mesh is loaded from an .obj file into vertex and index buffers and rendered using a standard `vkCmdDrawIndexed` call. In contrast, the tessellated version represents a shift to dynamic geometry. Here, the static mesh is replaced by a low-resolution, procedurally generated control grid and `vkCmdDraw` was used instead of `vkCmdDrawIndexed`. The graphics pipeline is fundamentally changed to incorporate Tessellation Control (TCS) and Tessellation Evaluation (TES) shaders, which dynamically generate high-detail geometry on the GPU. Consequently, the input topology is changed to `VK_PRIMITIVE_TOPOLOGY_PATCH_LIST`, and the index buffer is eliminated in favour of direct `vkCmdDraw` calls on the control patches. A second major implementation split is evident in the two approaches to stereo rendering. The single-pass stereo implementation leverages modern GPU hardware by enabling the Vulkan multiview feature. This is an efficient, hardware-level solution where the render pass (`VkRenderPassMultiviewCreateInfo`), swapchain images, and depth buffer are configured as array resources (`imageArrayLayers = 2`). This architecture allows the GPU to render both the left and right eye views in a single, parallelised rendering pass with just one `vkCmdDrawIndexed` call, using the `gl_ViewIndex` shader variable to select the appropriate view matrix. Conversely, the multi-pass stereo implementation adopts a software-based approach. Without using the hardware multiview feature, it modifies the `recordCommandBuffer` function to execute the rendering logic twice within a single frame. For each pass, it sets a different viewport for the left and right eyes and uses a push constant to send a `viewIndex` to the shader, effectively resulting in two distinct `vkCmdDrawIndexed` calls per frame. Finally, the multi-pass stereo with LOD implementation represents the architectural integration of multi-pass stereo and tessellation approaches. It merges the dynamic geometry pipeline with the multi-pass stereo rendering logic, where two

draw calls (`vkCmdDraw`) with different viewports and push constants are issued per frame to render the dynamically tessellated terrain for both eyes. This final version is the most computationally intensive features from the preceding stages into a single, comprehensive rendering pipeline.

4.3 Data Collection Setup

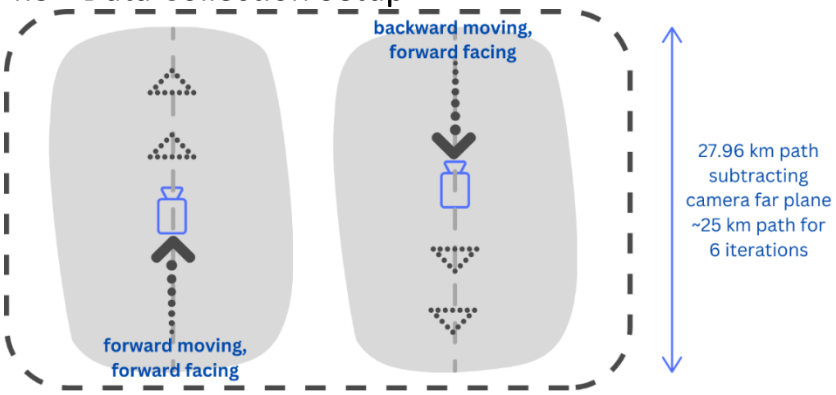


Figure 4-3 Camera traversal path for data collection

A systematic data collection methodology was established to ensure a fair and reproducible comparison between the different rendering configurations. The experimental setup focused on two main components: a standardised camera traversal to simulate a typical viewing scenario, and a logging mechanism to record performance data. The core of the data collection is a predefined, automated camera path designed to cover a significant portion of the Apollo 11 landing site. This path was implemented within the `updateUniformBuffer()` method, which updates the camera's position for each frame. The camera performs a linear traversal of approximately 25.5 km along the terrain's Y axis to generate results in Chapter 6. The camera moves forward along a predefined path until the terrain ahead approaches the far plane distance, illustrated in Figure 4-3. At this boundary, it reverses its movement to travel backwards, all while maintaining a forward-facing orientation. To gather a robust dataset, this traversal is repeated 6 times in a back-and-forth pattern (without camera rotation) to mitigate the impact of initial, non-representative frame times caused by uncached assets and allow performance to stabilise. Throughout this automated flight, two key metrics are continuously recorded for every frame: the application-level render time, the total time taken to produce a frame, and FPS. Once the camera completes all six traversals, a method is called to log the collected data in CSV files with a name including the current timestamp. This process prevents data from previous runs from being overwritten and ensures a clean dataset for each experimental configuration. After all performance logs have been successfully written to disk, the application programmatically signals the GLFW window to close, concluding the experimental session. This approach guarantees that each rendering technique is benchmarked under identical conditions, providing a solid foundation for the performance analysis presented in the Results and Discussion chapter.

Chapter 5 Implementation

This chapter provides a detailed account of the technical implementation of the high-fidelity lunar terrain renderer developed for this study. It begins with a foundational overview of the Vulkan API, outlining the fundamental workflow from instance creation and device setup to the main rendering loop that orchestrates drawing operations via command buffers. This is followed by a discussion on the role of shaders in Vulkan, which are responsible for the programmable state of the graphics pipeline and are compiled into the mandatory SPIR-V bytecode format for consistency and reduced driver overhead. The chapter then elaborates on the specific rendering implementations, starting with a baseline approach for rendering a static, pre-generated .obj mesh, which serves as the architectural foundation. Subsequently, it describes the evolution of this renderer into a dynamic Level of Detail (LOD) system that utilises hardware tessellation. This advanced technique replaces the static mesh with a simple control grid, dynamically generating high-fidelity geometry on the GPU based on camera distance and using a 16-bit heightmap for surface displacement. Next, the focus shifts to stereo rendering, a crucial component for simulating the demanding workloads of Virtual Reality applications. The implementation explores two distinct approaches: an efficient, hardware-accelerated single-pass technique using Vulkan's built-in multi-view feature to render both eye views in a single draw call, and a software-driven multi-pass method that manually renders the scene twice per frame. The chapter then details the integration of the multi-pass stereo approach with the tessellation-based LOD system, representing the most computationally complex rendering scenario. Furthermore, the implementation details the integration of the OpenXR SDK to enable a PC-tethered VR viewing, explaining the architectural changes required to support a Head-Mounted Display alongside the desktop mirror view. Finally, the chapter concludes by outlining the implementation for systematic data collection, detailing how performance metrics like render time and FPS were logged during automated camera traversals for the analysis presented in the following chapter. The implementation is available in the GitHub repository (Albayrak, 2025b).

5.1 Vulkan API Overview

Vulkan is a cross-platform, low-level graphics API designed to address the limitations of older graphics APIs like OpenGL and DirectX 11. It provides explicit control over GPU resources and operations for developers, reducing CPU overhead and enabling multi-threading. This explicit control further covers memory management, resource binding and synchronisation, i.e. fences are designed for GPU to CPU synchronisation while semaphores synchronise GPU tasks. Vulkan also regulates shader compilation through bytecode format, SPIR-V (Standard Portable Intermediate Representation). To illustrate the fundamental workflow and the complexity of

Vulkan, the following 8 steps are explained and illustrated in Figure 5-1. This subsection is explained by using Vulkan resources (Khronos, 2025a).

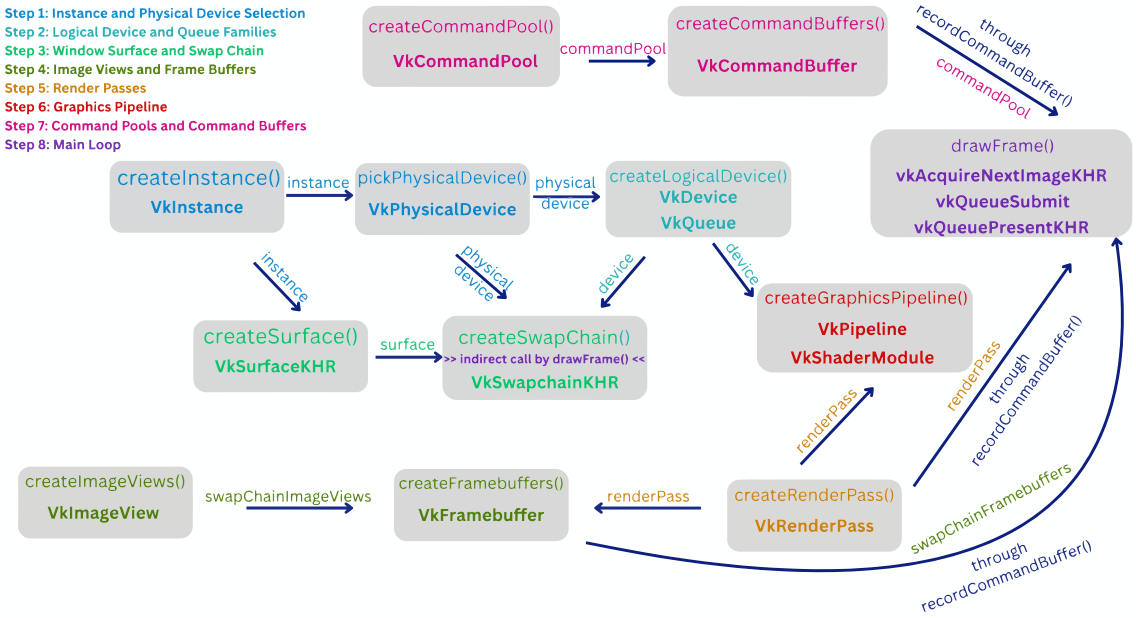


Figure 5-1 An overview for simple rendering using Vulkan

1. A Vulkan application begins with creating a `VkInstance`, describing the application and its extensions. Subsequently, the application sets `VkPhysicalDevices`, supported hardware e.g. a graphics card, based on VRAM size and capabilities.
2. After selecting a physical device, a `VkDevice` (logical device) is created. This step determines which `VkPhysicalDeviceFeatures` will be utilised (e.g. multi-viewport rendering). It also defines `VkQueue` families, which allocate queues for asynchronous operations such as graphics, compute, or memory transfer commands.
3. For rendering to a display, `VkSurfaceKHR`, created via a Window System Interface (WSI) extension, is needed to abstract the window (i.e. platform-specific window). A `VkSwapchainKHR` is then created, acting as a collection of render targets to ensure that only complete frames are presented to the screen, preventing visual artefacts.
4. The frames acquired from the swap chain for rendering are wrapped into `VkImageView` and `VkFramebuffer`. An image view addresses a specific portion of an image, while a framebuffer binds image views to be used as colour, depth, and stencil targets for rendering operations.
5. A `VkRenderPass` object describes the types of images (e.g. colour image, depth, stencil) used during rendering, their usage (e.g. a single colour target), and how their contents should be treated (e.g. cleared to a solid colour before drawing).
6. The `VkPipeline` object configures the entire graphics pipeline, including both configurable states (like viewport size and depth buffer operations) and programmable

states defined by `VkShaderModule` objects (created from standardized shader byte code). Most of this configuration is set in advance, offering driver optimisation opportunities.

7. Operations like drawing are first recorded into `VkCommandBuffer`, which are allocated from a `VkCommandPool` associated with a specific queue family. These command buffers detail sequences such as beginning a render pass, binding the graphics pipeline and issuing draw commands. Command buffers are typically recorded for each readily available swap chain image.
8. The main loop of the application orchestrates the rendering pipeline by first obtaining an image from the swap chain (`vkAcquireNextImageKHR`). The appropriate command buffer for that image is then submitted to a queue for execution (`vkQueueSubmit`). Finally, the rendered image is returned to the swap chain for presenting (`vkQueuePresentKHR`), with synchronisation objects (like semaphores), ensuring the correct order of asynchronous operations. This structure provides fine-grained control over the rendering process, which is critical for performance-intensive applications.

5.2 Shaders in Vulkan

In Vulkan, the programmable state of the graphics pipeline is configured using `VkShaderModule` objects, which are derived from shader bytecode. To render basic geometry, such as a triangle, both a vertex shader and a fragment shader are essential components. Shaders within Vulkan, and specifically in their SPIR-V (Standard Portable Intermediate Representation) format, can accommodate multiple entry points within a single file. This capability is particularly advantageous for complex pipelines, such as ray-tracing, which may involve numerous small shaders. A fundamental distinction in Vulkan's shader architecture, compared to preceding graphics APIs, is the mandatory use of a bytecode format, specifically SPIR-V, for shader code. SPIR-V functions as a binary intermediate representation for both graphical shader stages and compute kernels. This design choice was implemented to mitigate challenges encountered with human-readable shader languages like GLSL and HLSL. By adopting a standardized bytecode format and a unified compiler, Vulkan significantly enhances consistency in shader compilation and reduces driver overhead. Despite SPIR-V being a bytecode, developers typically do not compose it manually. Instead, high-level shading languages, such as GLSL (which is explicitly noted as being converted to SPIR-V for this thesis), are compiled into SPIR-V binaries. This process ensures adherence to standards and generates a single, portable binary that can be shipped with applications. The broader SPIR-V ecosystem also offers a variety of tools for generating this bytecode. Once compiled, the SPIR-V bytecode is loaded into the application. This process typically involves reading the binary data from a file into a buffer. This raw bytecode is then encapsulated within a `VkShaderModule` object, which serves as a lightweight wrapper around the shader bytecode. These shader modules are

subsequently integrated into the graphics pipeline during its creation phase. The actual compilation and linking of the SPIR-V bytecode into machine executable code for the GPU occurs when the graphics pipeline is created, allowing shader modules to be safely destroyed immediately after pipeline creation is complete. This subsection is explained by using Vulkan resources (Khronos, 2025b; Vulkan, 2025).

5.3 Implementation Overview

Table 5 1 briefly introduces the key architectural differences between implementation variations studied in this thesis. The discussion will begin with the foundational baseline implementation for rendering static meshes, then detailing its evolution into a dynamic Level of Detail (LOD) system with hardware tessellation and culminating in its adaptation for stereo rendering through both efficient single-pass and software-driven multi-pass techniques.

Table 5-1 Key differences between implementation variations

	.obj rendering	**LOD with tessellation**	**single-pass stereo**	**multi-pass stereo**	**multi-pass stereo with tessellation**
Geometry	Pre-loaded .obj mesh	Procedurally generated 2D grid of vertices	Pre-loaded .obj mesh	Pre-loaded .obj mesh	Procedurally generated 2D grid of vertices
Shaders	Vertex, Fragment	Vertex, Fragment, Tessellation Control & Evaluation	Vertex, Fragment	Vertex, Fragment	Vertex, Fragment, Tessellation Control & Evaluation
Special Vulkan Features	-	Tessellation Shader enabled	Multi View enabled	-	Tessellation Shader enabled
View/Proj Handling	Standard `ubo.proj * ubo.view`	Similar to .obj rendering in Tessellation Eval. (TES)	`gl_ViewIndex` for view/proj from UBO arrays	`pushConstants.viewIndex` for view/proj from UBO arrays	`pushConstants.viewIndex` for view/proj from UBO arrays
Texture Usage	-	16-bit heightmap for displacement in TES	-	-	16-bit heightmap for displacement in TES
Stereo Rendering	No	No	**Single-pass (Multiview)**: Renders both eyes in a single draw call.	**Multi-pass:** Renders each eye sequentially in separate draw calls.	**Multi-pass:** Renders each eye sequentially in separate draw calls.

5.4 Mesh (.obj) Rendering

This implementation serves as a baseline, rendering a pre-existing .obj mesh without advanced features like tessellation or stereo rendering. It is responsible for loading and rendering a pre-generated, high-resolution static mesh of the lunar terrain from a Wavefront .obj file. The mesh generation part is explained in section 3.2. This implementation heavily follows the architecture shown in Figure 4-2 and the steps explained in Figure 5-1. Major implementation steps are given below:

1. Model Loading: The `loadModel()` loads vertex data (positions, colours, normals) and index data from the specified .obj file. Unlike the tessellated versions, this uses an index buffer to avoid duplicating vertex data when multiple faces share vertices.
2. Vertex Structure: The Vertex struct defines the attributes passed to the vertex shader: position, colour, and normal.
3. Buffer Creation: `createVertexBuffer()` and `createIndexBuffer()` allocate device-local buffers and transfer vertex and index data from host-visible staging buffers to GPU memory.
4. Graphics Pipeline: The `createGraphicsPipeline()` function sets up a standard graphics pipeline with two shader stages: a vertex shader and a fragment shader. The input assembly uses `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST` for drawing triangles from the loaded mesh.
5. Descriptor Set Layout: The `createDescriptorSetLayout()` defines a uniform buffer (`uboLayoutBinding`) bound at `binding = 0` for vertex shader access `VK_SHADER_STAGE_VERTEX_BIT`. No image sampler is configured here as the default shaders do not use textures. However, it's used for tessellation scenario, for processing the heightmap texture.
6. Uniform Buffer Object (UBO): `UniformBufferObject` (UBO) for this pipeline is simple, containing single `glm::mat4` members for the model, view, and projection matrices, along with the camera's position.
7. Drawing: The core of the rendering process happens in `recordCommandBuffer()` which executes once per frame. This function binds the graphics pipeline and the vertex buffers and index buffers and then issues a single `vkCmdDrawIndexed` call to render the entire terrain mesh.

Fragment Shader: This shader receives the normalized values from the vertex shader. It performs a standard Phong lighting calculation, using the interpolated normal, world position, and view direction to compute ambient, diffuse, and specular lighting components. These are combined to produce the final pixel colour.
Fragment shader is the same for all distinct implementations.

```
vec3 normal = normalize(fragNormal);
vec3 lightDir = normalize(vec3(0.0, 4.0,
0.0)); // arbitrary direction
vec3 viewDir = normalize(fragViewDir);
vec3 reflectDir = reflect(-lightDir,
normal);

float diff = max(dot(normal, lightDir),
0.0);
    float spec = pow(max(dot(viewDir,
reflectDir), 0.0), 32.0);

vec3 ambient = vec3(0.05);
vec3 diffuse = diff * vec3(0.25);
vec3 specular = spec * vec3(0.4);
vec3 finalColour = ambient + diffuse +
specular;

outColor = vec4(finalColour, 1.0);
```

Vertex Shader: This shader's primary function is to transform the incoming vertex positions from model space to clip space. It receives vertex data and accesses the UBO for the transformation matrices (`ubo.proj * ubo.view * ubo.model`). It calculates world-space values for the vertex position and normal and computes the view direction vector, passing these values to the fragment shader for lighting.

```
layout(binding = 0) uniform
UniformBufferObject {
    mat4 model;
    mat4 view;
    mat4 proj;
    vec4 cameraPosition;
} ubo;

layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inColor;
layout(location = 2) in vec3 inNormal;
layout(location = 0) out vec3 fragNormal;
layout(location = 1) out vec3 fragViewDir;

void main() {
    vec3 worldPos = vec3(ubo.model *
vec4(inPosition, 1.0));
    fragNormal = normalize(mat3(ubo.model) *
inNormal);
    fragViewDir =
normalize(ubo.cameraPosition.xyz - worldPos);
    gl_Position = ubo.proj * ubo.view *
vec4(worldPos, 1.0);
}
```

5.5 Camera Distance Based LOD with Tessellation Rendering

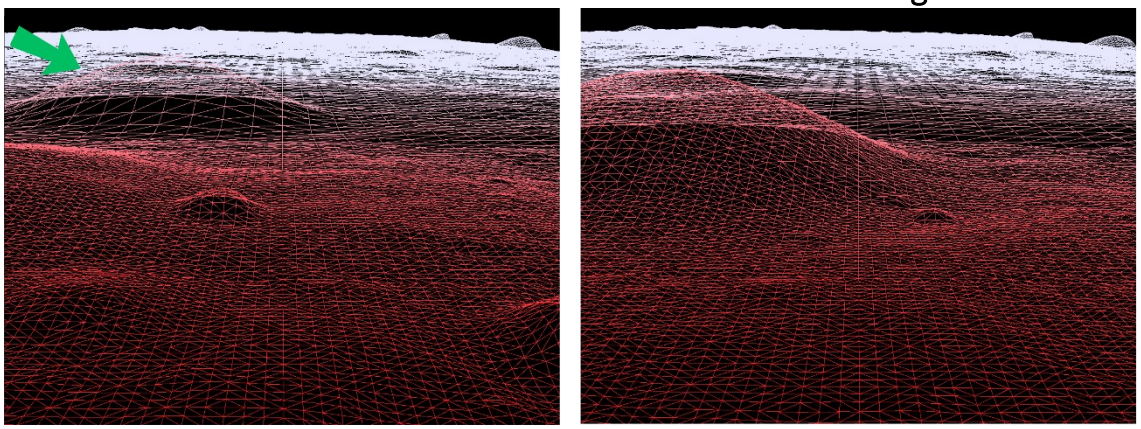


Figure 5-2 An illustration of geometry detailing by camera distance
It shows how surface details are enhanced by camera distance (closer distances with higher details). The camera distance and tessellation parameters were changed in this figure for better visualisation.

This implementation adopts an LOD approach using hardware tessellation based on camera distance, offering dynamic mesh refinement. Figure 5-2 shows how geometry is more detailed for the regions closer to the camera, note that camera distance and tessellation level parameters were changed for better visualisation for this figure. This approach significantly reduces the initial memory and processing load by starting with a simple base mesh instead of a multi-million-triangle .obj file. The `loadModel()` function doesn't load a .obj file. Instead, it procedurally generates a low-resolution control grid of quad patches that covers the terrain area. Consequently, an index buffer is no longer needed. Some changes in the `C++` part are:

- Device Features: The `createLogicalDevice()` function explicitly enables the tessellation shader feature on the physical device (`deviceFeatures.tessellationShader = VK_TRUE`).
- Instead of pre-loaded .obj mesh rendered with `vkCmdDrawIndexed` to a dynamically generated mesh using tessellation shaders and `vkCmdDraw` in `recordCommandBuffer()`.
- This implementation loads a 16-bit single-channel heightmap in `createTextureImage()`. This heightmap provides the elevation data that the tessellation evaluation shader uses for displacement.
- The `UniformBufferObject` (UBO) now includes `minZ` and `maxZ` to scale the heightmap values correctly. These `minZ` and `maxZ` values are the displacement range and are sent to shaders with camera parameters.
- The input assembly state is changed from `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST` to `VK_PRIMITIVE_TOPOLOGY_PATCH_LIST` to signal to Vulkan that the pipeline is using quad patches for tessellation.

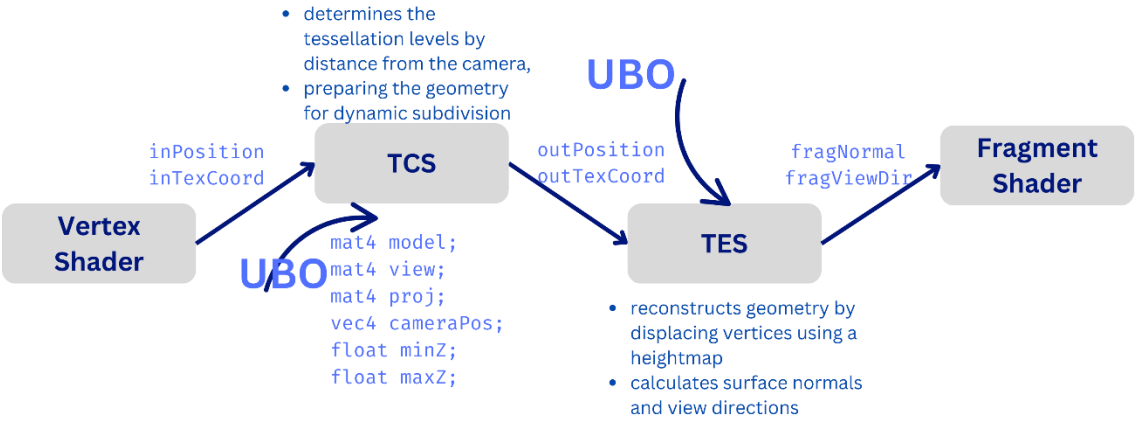


Figure 5-3 An overview of shaders used for LOD by camera distance

The `createGraphicsPipeline()` function now loads and includes four shader stages: vertex shader, tessellation control shader (TCS), tessellation evaluation shader (TES), and fragment shader. Also, `VkPipelineTessellationStateCreateInfo` struct is added to the `createGraphicsPipeline()` method, specifying the number of control points per patch (`patchControlPoints = 4` for quads). Functionalities of these shaders:

- Vertex shader: This is a minimal vertex shader for tessellation. It takes `inPosition` and `inTexCoord` and directly passes them as `outPosition` and `outTexCoord` to the TCS.
- Tessellation Control Shader
    - It receives the four control points of a patch from the vertex shader. For each patch, it calculates the distance from the camera to its vertices in view space.

- It calculates the distance squared of each control point of the patch from the camera using the `ubo.view` matrix.
- Based on this distance, it determines how finely the patch should be subdivided by setting the `gl_TessLevelOuter` and `gl_TessLevelInner` output variables.
- The primary role of the TCS is to determine these tessellation levels (`gl_TessLevelOuter` and `gl_TessLevelInner`). These levels control how many times each edge and the interior of the patch will be subdivided.
- `MAX_TESS_LEVEL` is applied for close distances, and `MIN_TESS_LEVEL` for far distances. Inner tessellation levels are set based on the maximum of adjacent outer levels. This creates the dynamic LOD, with more detail closer to the camera.

- Tessellation Evaluation Shader
  - After the hardware tessellator generates new vertices based on the tessellation levels, this shader is invoked for each new vertex. It uses the vertex's barycentric coordinates (`gl_TessCoord`) to interpolate a position on the base patch.
  - `basePos.z` is interpolated by `mix(ubo.minZ, ubo.maxZ, heightSample)`, effectively creating the terrain's height variations based on the heightmap data. It calculates `worldPos` from `basePos`, then `fragPosition` and `fragViewDir` (from `cameraPosition` to `worldPos`).
  - The `calculateTangentSpaceNormal()` function samples the heightmap at adjacent texel coordinates (`hL, hR, hD, hU`) to compute a normal vector in tangent space, then transforms it to world space for `fragNormal`. This ensures normals correctly reflect the tessellated surface.
- Fragment shader: It receives `fragNormal` and `fragViewDir` from the TES. It applies the same basic Phong lighting model to produce the output colour. *Fragment shader is the same for all distinct implementations.*

```
// TCS Tessellation Control Shader
// UniformBufferObject (UBO) shown in Figure 5-3 is also an input here

const float MIN_DISTANCE = 1.0;
const float MAX_DISTANCE = 2000.0;
const float MIN_TESS_LEVEL = 1.0;
const float MAX_TESS_LEVEL = 8.0;
void main()
{
    outPosition[gl_InvocationID] = inPosition[gl_InvocationID];
    outTexCoord[gl_InvocationID] = inTexCoord[gl_InvocationID];
    gl_out[gl_InvocationID].gl_Position =
vec4(inPosition[gl_InvocationID], 1.0);

    if (gl_InvocationID == 0)
    {
        const float MIN_DISTANCE_SQ = MIN_DISTANCE * MIN_DISTANCE;
        const float MAX_DISTANCE_SQ = MAX_DISTANCE * MAX_DISTANCE;

        vec4 v0 = ubo.view * vec4(inPosition[0], 1.0);
        vec4 v1 = ubo.view * vec4(inPosition[1], 1.0);
        vec4 v2 = ubo.view * vec4(inPosition[2], 1.0);
        vec4 v3 = ubo.view * vec4(inPosition[3], 1.0);

        float d0_sq = dot(v0.xyz, v0.xyz);
        float d1_sq = dot(v1.xyz, v1.xyz);
        float d2_sq = dot(v2.xyz, v2.xyz);
        float d3_sq = dot(v3.xyz, v3.xyz);

        float dist0 = clamp((d0_sq - MIN_DISTANCE_SQ) / (MAX_DISTANCE_SQ
- MIN_DISTANCE_SQ), 0.0, 1.0);
        float dist1 = clamp((d1_sq - MIN_DISTANCE_SQ) / (MAX_DISTANCE_SQ
- MIN_DISTANCE_SQ), 0.0, 1.0);
        float dist2 = clamp((d2_sq - MIN_DISTANCE_SQ) / (MAX_DISTANCE_SQ
- MIN_DISTANCE_SQ), 0.0, 1.0);
        float dist3 = clamp((d3_sq - MIN_DISTANCE_SQ) / (MAX_DISTANCE_SQ
- MIN_DISTANCE_SQ), 0.0, 1.0);

        float tessLevel0 = mix(MAX_TESS_LEVEL, MIN_TESS_LEVEL,
min(dist2, dist0));
        float tessLevel1 = mix(MAX_TESS_LEVEL, MIN_TESS_LEVEL,
min(dist0, dist1));
        float tessLevel2 = mix(MAX_TESS_LEVEL, MIN_TESS_LEVEL,
min(dist1, dist3));
        float tessLevel3 = mix(MAX_TESS_LEVEL, MIN_TESS_LEVEL,
min(dist3, dist2));

        gl_TessLevelOuter[0] = tessLevel0;
        gl_TessLevelOuter[1] = tessLevel1;
        gl_TessLevelOuter[2] = tessLevel2;
        gl_TessLevelOuter[3] = tessLevel3;

        gl_TessLevelInner[0] = max(tessLevel1, tessLevel3);
        gl_TessLevelInner[1] = max(tessLevel0, tessLevel2);
    }
}
```

```
// TES Tessellation Evaluation Shader
// UniformBufferObject (UBO) shown in Figure 5 3 is also an input here

layout(binding = 1) uniform sampler2D heightMap;
vec3 calculateTangentSpaceNormal(vec2 texCoord)
{
    vec2 offset = 1.0 / textureSize(heightMap, 0);
    float hL = texture(heightMap, texCoord - vec2(offset.x, 0)).r;
    float hR = texture(heightMap, texCoord + vec2(offset.x, 0)).r;
    float hD = texture(heightMap, texCoord - vec2(0, offset.y)).r;
    float hU = texture(heightMap, texCoord + vec2(0, offset.y)).r;
    float scale = ubo.maxZ - ubo.minZ;
    vec3 N = vec3(scale * (hL - hR), scale * (hD - hU), 2.0);
    return normalize(N);
}

void main()
{
    float u = gl_TessCoord.x;
    float v = gl_TessCoord.y;
    vec2 texCoord = mix(mix(inTexCoord_TES[0], inTexCoord_TES[1], u),
mix(inTexCoord_TES[2], inTexCoord_TES[3], u), v);
    vec3 basePos = mix(mix(inPosition_TES[0], inPosition_TES[1], u),
mix(inPosition_TES[2], inPosition_TES[3], u), v);

    float heightSample = texture(heightMap, texCoord).r;
    basePos.z += mix(ubo.minZ, ubo.maxZ, heightSample);

    vec3 worldPos = vec3(ubo.model * vec4(basePos, 1.0));
    fragPosition = worldPos;

    fragViewDir = normalize(ubo.cameraPosition.xyz - worldPos);

    vec3 tangentNormal = calculateTangentSpaceNormal(texCoord);
    fragNormal = normalize(mat3(transpose(inverse(ubo.model))) *
tangentNormal);

    gl_Position = ubo.proj * ubo.view * vec4(worldPos, 1.0);
}
```

5.6 An Overview of Stereo Rendering

Stereo rendering is the process of generating two distinct images of a scene from slightly different viewpoints, simulating the separation between the human eyes. The figure illustrates the left eye and right eye rendering with a slight disparity (Blondé et al., 2010). It is used to create the illusion of depth and facilitate 3D viewing, particularly crucial for VR applications. It involves rendering a scene from two slightly different viewpoints, one for each eye, to simulate human binocular vision. This process increases the computational workload compared to mono rendering, as the scene must effectively be processed twice. Maintaining a high and stable frame rate is vital for a comfortable VR experience, minimising motion sickness and other undesirable effects (Lai et al., 2017; Jansen et al., 2023). Single-pass stereo and multi-pass stereo approaches can be adopted for stereo rendering in the Vulkan API (Krupip, 2019). To this end, we created 2 renderings, i.e. the left camera and the right camera, with a horizontal distance of 6.5 cm, as the interpupillary distance is close to this value (Lee et al., 2022). *However, delivering an optimal stereo viewing is out of this study's scope; the main goal of this stereo rendering is to analyse its rendering performance, as it may cause extra workload for CPU (Wong, 2016).*

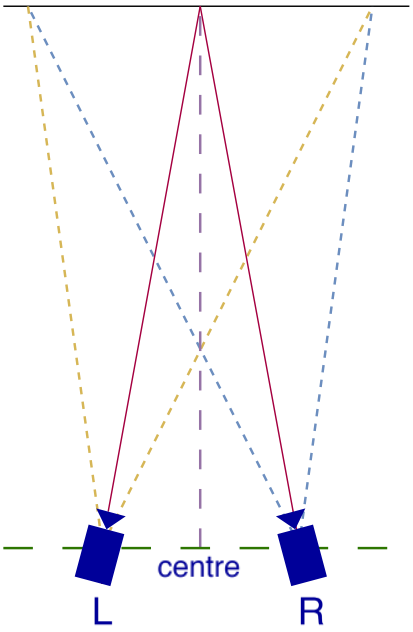


Figure 5-4 An illustration of off-axis camera layout for stereo viewing

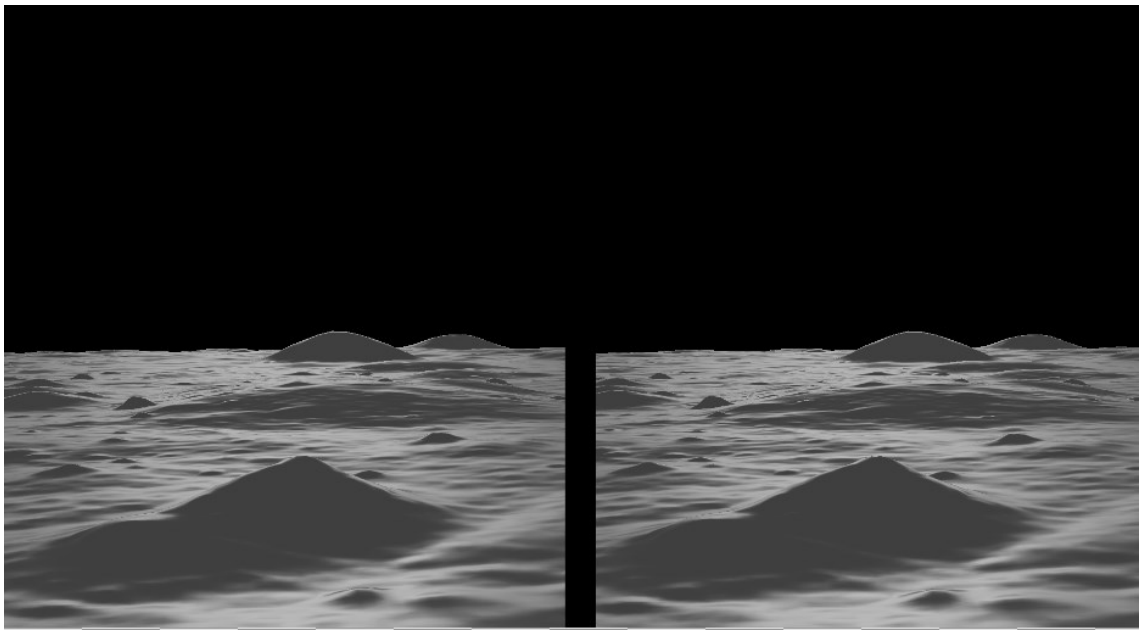


Figure 5-5 Examples of the left eye and the right eye rendering for the same frame

5.7 Single-Pass Stereo Rendering

This implementation runs stereo rendering by leveraging Vulkan's multi-view feature. Converting the wavefront .obj mesh rendering to single-pass stereo involves crucial changes in the Vulkan API level, primarily focusing on arrayed resources and render pass configuration.

- The most critical change is enabling the multi-view feature by setting `multiviewFeatures.multiview = VK_TRUE` when creating the logical device. This feature allows the GPU to process multiple views simultaneously.
- The swap chain is configured with `imageArrayLayers = 2` to create two layers for the stereo output (one for each eye). Similarly, `VkImageView` and `VkImage` creation methods are updated to accept and utilise a `layerCount` of 2, creating 2D array images and image views. The depth buffer is also created as an arrayed resource with `arrayLayers = 2` to match the swap chain images.
- A `VkRenderPassMultiviewCreateInfo` structure is added to the `VkRenderPassCreateInfo`, defining a `viewMask = 0b00000011`, enabling view 0 (left camera view) and view 1 (right camera view).
- `UniformBufferObject` is modified to hold arrays of matrices `(view[2], proj[2])`, and the `updateUniformBuffer` function is updated to calculate and store the transformation matrices for both the left and right eyes.
- The vertex shader uses the built-in GLSL variable `gl_ViewIndex` to index into the `view`, `projection` and `cameraPosition` arrays, effectively applying the correct eye-specific matrices and camera position for each view layer within a single shader execution.

```
// vertex shader
#extension GL_EXT_multiview : require

layout(binding = 0) uniform UniformBufferObject {
    mat4 model;
    mat4 view[2];
    mat4 proj[2];
    vec4 cameraPosition[2];
} ubo;

// .. similar input/output with no stereo version

void main() {
    fragNormal = normalize(inNormal);
    fragPosition = vec3(ubo.model * vec4(inPosition, 1.0));
    fragViewDir = normalize(ubo.cameraPosition[gl_ViewIndex].xyz -
fragPosition);
    gl_Position = ubo.proj[gl_ViewIndex] * ubo.view[gl_ViewIndex]
                    * ubo.model * vec4(inPosition, 1.0);
}
```

```
// code piece from updateUniformBuffer(uint32_t currentImage) method
// showing different camera view and projection settings for the left and
// the right eye

// stereo parameters
float ipd = 0.064f;
float centre = ipd * 0.5f;

constexpr float fov = glm::radians(30.0f);
float aspectRatio = swapChainExtent.width / (float)swapChainExtent.height;
float nearPlane = 0.1f;
float farPlane = 4000.0f; // Increased from 1000.0f
glm::vec3 rightDir = glm::normalize(glm::cross(forwardDirection,
glm::vec3(0.0f, 0.0f, 1.0f)));

// frustum parameters
float top = nearPlane * tan(fov / 2.0f);
float bottom = -top;
float right = top * aspectRatio;
float left = -right;
float frustumShift = centre * nearPlane / glm::length(forwardDirection);

// left eye position
glm::vec3 leftEyePosition = cameraPosition - rightDir * centre;
glm::vec3 leftViewDir = glm::normalize(cameraTarget - leftEyePosition);
ubo.view[0] = glm::lookAt(leftEyePosition, leftEyePosition +
forwardDirection, upVector);
ubo.proj[0] = glm::frustum(left - frustumShift, right - frustumShift,
bottom, top, nearPlane, farPlane);
ubo.proj[0][1][1] *= -1;
ubo.cameraPosition[0] = glm::vec4(leftEyePosition, 1.0f);

// right eye position
glm::vec3 rightEyePosition = cameraPosition + rightDir * centre;
glm::vec3 rightViewDir = glm::normalize(cameraTarget - rightEyePosition);
ubo.view[1] = glm::lookAt(rightEyePosition, rightEyePosition +
forwardDirection, upVector);
ubo.proj[1] = glm::frustum(left + frustumShift, right + frustumShift,
bottom, top, nearPlane, farPlane);
ubo.proj[1][1][1] *= -1;
ubo.cameraPosition[1] = glm::vec4(rightEyePosition, 1.0f);
```

5.8 Multi-Pass Stereo Rendering

Multi-pass stereo implementation provides an alternative, software-based approach to stereo rendering. Instead of relying on the hardware multi-view feature, it manually renders the scene twice within a single frame, once for each eye. Consequently, the multi-view feature is not explicitly enabled when creating the logical device.

- The `UniformBufferObject` in C++ still includes arrays for `view`, `proj`, and `cameraPosition` to store separate data for each eye, similar to the single-pass approach.
- In `createGraphicsPipeline()`, a `VkPushConstantRange` is added to the `VkPipelineLayoutCreateInfo`. Push constants are an efficient way to send a small amount of data to shaders for a specific draw call. Here, we use it to send the eye index (0 for left, 1 for right).

- `recordCommandBuffer()` executes two separate `vkCmdDrawIndexed` calls within the same render pass. For each eye (left and right), a distinct viewport and scissor rectangle are set (`vkCmdSetViewport, vkCmdSetScissor`) to render into specific halves of the overall window.
- The window `WIDTH` is adjusted to 1800 + 50 pixels to accommodate two side-by-side views with a gap (see Figure 5-5), for illustration purposes.
- The vertex shader does not use `gl_ViewIndex`. Instead, it relies on a push constant `viewIndex` passed from the C++ application to select the correct view and projection matrices from the UBO.

```
// vertex shader
layout(push_constant) uniform PushConstants {
    int viewIndex;
} pushConstants;

// The UBO structure remains the same
// .. similar input/output with no stereo version

void main() {
    fragNormal = normalize(inNormal);
    fragPosition = vec3(ubo.model * vec4(inPosition, 1.0));

    // Use the viewIndex from the push constant instead of gl_ViewIndex
    gl_Position = ubo.proj[pushConstants.viewIndex] *
ubo.view[pushConstants.viewIndex] * ubo.model * vec4(inPosition, 1.0);
}
```

```
// code piece from recordCommandBuffer(VkCommandBuffer commandBuffer,
uint32_t imageIndex)
// render left eye, similar apply to the right eye with viewIndex = 1
{
    VkViewport viewport{};
    viewport.x = 0.0f;
    viewport.y = 0.0f;
    viewport.width = (float)swapChainExtent.width / 2.0f; // Half left
width
    viewport.height = (float)swapChainExtent.height;
    viewport.minDepth = 0.0f;
    viewport.maxDepth = 1.0f;
    vkCmdSetViewport(commandBuffer, 0, 1, &viewport);

    VkRect2D scissor{};
    scissor.offset = { 0, 0 };
    scissor.extent = { swapChainExtent.width / 2, swapChainExtent.height };
    vkCmdSetScissor(commandBuffer, 0, 1, &scissor);

    // Push the view index for the left eye (0)
    int viewIndex = 0;
    vkCmdPushConstants(
        commandBuffer,
        pipelineLayout,
        VK_SHADER_STAGE_VERTEX_BIT,
        0,
        sizeof(int),
        &viewIndex);

    vkCmdDrawIndexed(commandBuffer, static_cast<uint32_t>(indices.size()),
1, 0, 0, 0);
}
```

5.9 Multi-Pass Stereo Rendering with Tessellation

This implementation combines the dynamic LOD capabilities of tessellation with the multi-pass stereo rendering approach. It inherits the entire tessellation pipeline (vertex, tessellation, and fragment shaders), procedural mesh generation and incorporates the multi-pass stereo rendering logic.

- Similar to the multi-pass stereo rendering, the `recordCommandBuffer` function performs two distinct `vkCmdDraw` calls (instead of `vkCmdDrawIndexed` due to tessellation). Each draw call configures its own viewport and scissor rectangle to render into half of the window and uses `vkCmdPushConstants` to send the appropriate `viewIndex` to the shader stages.
- `VkPushConstantRange` is defined in the pipeline layout to send the `viewIndex` to all relevant shader stages `(VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT | VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT)`.
- The TCS shader calculates tessellation levels based on camera distance, yet it uses `ubo.view[pushConstants.viewIndex]` to ensure the LOD calculation is performed relative to the specific eye currently being rendered.
- The TES shader performs displacement mapping using the 16-bit heightmap, as explained in section 5.5. It also relies on `pushConstants.viewIndex` to correctly determine `fragViewDir` from the active eye's `ubo.cameraPosition`.

```
// TCS Tessellation Control Shader
layout(push_constant) uniform PushConstants {
    int viewIndex;
} pushConstants;

layout(vertices = 4) out;

layout(location = 0) in vec3 inPosition[];
layout(location = 1) in vec2 inTexCoord[];

layout(location = 0) out vec3 outPosition[];
layout(location = 1) out vec2 outTexCoord[];

layout(std140, binding = 0) uniform UniformBufferObject {
    mat4 model;
    mat4 view[2];
    mat4 proj[2];
    vec4 cameraPosition[2];
    vec4 minMaxZ;
} ubo;

// below code is similar to Tessellation Control Shader in section 5.5
// except ubo.view is an array,
// it uses ubo.view[pushConstants.viewIndex]
//
```

```
// TES Tessellation Evaluation Shader
layout(push_constant) uniform PushConstants {
    int viewIndex;
} pushConstants;

layout(quads) in;
layout(location = 0) in vec3 inPosition_TES[];
layout(location = 1) in vec2 inTexCoord_TES[];

layout(location = 0) out vec3 fragNormal;
layout(location = 2) out vec3 fragViewDir;

layout(std140, binding = 0) uniform UniformBufferObject {
    mat4 model;
    mat4 view[2];
    mat4 proj[2];
    vec4 cameraPosition[2];
    vec4 minMaxZ;
} ubo;
// below code is similar to Tessellation Evaluation Shader in section 5.5
// except inPosition, inTexCoord, are converted to arrays
// like inPosition_TES, inTexCoord_TES, size of 2 (0 for left, 1 for right)
```

5.10 Integrating OpenXR into the Vulkan Renderer for VR Viewing

This section explains the implementation that supports PC-tethered virtual reality (VR) views, enabling the high-fidelity lunar terrain to be streamed to and experienced on a modern VR headset. PC-tethered VR (as introduced in section 2.3) is an architectural approach where a VR display is connected to a powerful computer, which handles all computationally intensive rendering and simulation tasks. This contrasts with standalone VR systems, which directly integrate all necessary processing hardware into the headset. Standalone headsets are constrained by mobile processors, limited memory, and integrated GPUs, which require significant performance optimisations, often at the cost of visual fidelity. By offloading the rendering workload to a PC's high-end CPU and GPU, PC-tethered VR can achieve far superior graphical quality, supporting environments with higher polygon counts, complex dynamic lighting, and advanced post-processing effects (Lai et al., 2017; Rendevski et al., 2022). The output of this implementation is streamed into the Meta Quest 3 device by using the Meta Quest Link desktop app through a wired connection. To enable VR capabilities, OpenXR SDK, an open, royalty-free standard from the Khronos Group, is integrated into the rendering implementation with the Vulkan API. OpenXR delivers a single, unified API that can run on any compliant hardware from vendors like Meta, HTC, Microsoft, and Valve, with the vendor's specific OpenXR runtime translating the standard API calls into hardware-specific commands (Khronos, 2019). Thus, by targeting the OpenXR API, this implementation is not tied to a specific headset like the Meta Quest 3. It can function with any OpenXR-compliant HMD that has a compatible runtime installed on the host PC. Figure 5-6 shows a partial call graph of the rendering implementation regarding OpenXR additions. The full implementation can be represented by merging Figure 4-2 and Figure 5-6. Implementation details are given below:

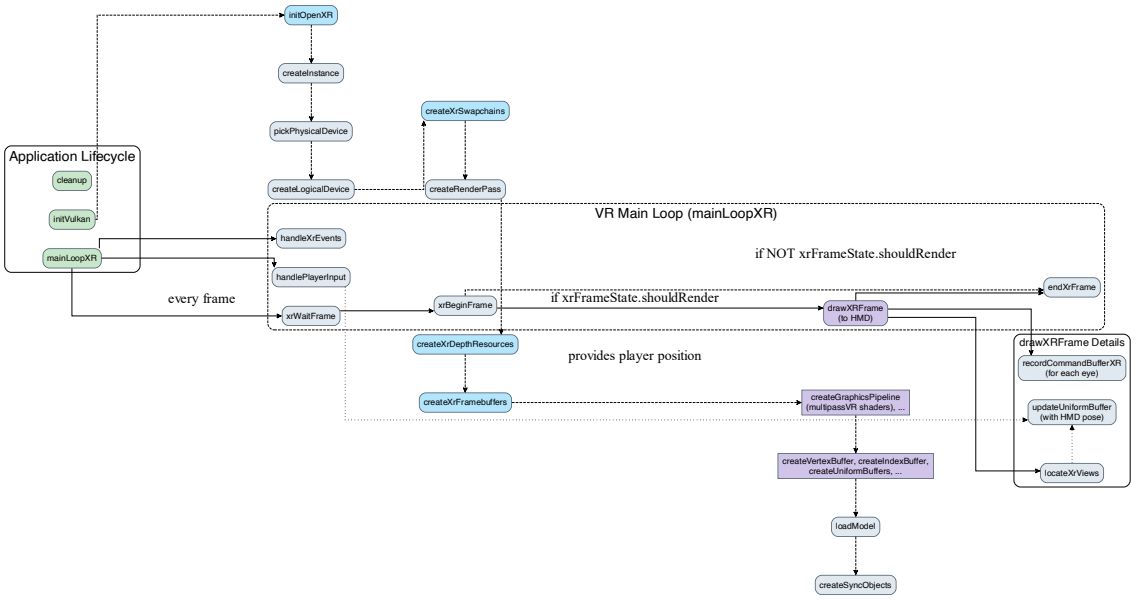


Figure 5-6 The call graph of the OpenXR-Vulkan rendering architecture

- `initOpenXR()` is called first. It generates an `XrApplicationInfo` struct and an `XrInstanceCreateInfo` struct, requesting the essential `XR_KHR_VULKAN_ENABLE_EXTENSION_NAME` extension. A call to `xrCreateInstance` establishes a connection with the active OpenXR runtime. It then retrieves crucial function pointers for Vulkan integration, such as `xrGetVulkanInstanceExtensionsKHR` and `xrGetVulkanGraphicsDeviceKHR`.
- `createInstance()` is different from the non-VR version. Instead of only getting extensions from `GLFW`, it now queries both `GLFW` and OpenXR. It calls `xrGetVulkanInstanceExtensionsKHR` to get required Vulkan instance extensions, which are then parsed and added to the list of extensions requested by `GLFW`. The resulting merged list is used to create a single `VkInstance` that supports both rendering targets.
- `pickPhysicalDevice()` asks the OpenXR runtime to provide the required `VkPhysicalDevice` by calling `xrGetVulkanGraphicsDeviceKHR`. This guarantees that the chosen GPU is compatible with the VR hardware and runtime.
- `createLogicalDevice()` is the main bridge between Vulkan and OpenXR. It queries OpenXR for required device-level Vulkan extensions via `xrGetVulkanDeviceExtensionsKHR`. This is crucial for enabling features like multiview (`VK_KHR_multiview`). These extensions are added to the application's own required list (`VK_KHR_SWAPCHAIN_EXTENSION_NAME`). After the `VkDevice` is created with the combined extensions and required features (like `multiviewFeatures`), it is immediately used to create the `XrSession`. This is achieved by yielding an `XrGraphicsBindingVulkanKHR` structure with the handles of our application's

VkInstance, VkPhysicalDevice, VkDevice, and the graphics queue index. Passing this binding to xrCreateSession explicitly links our Vulkan context to the OpenXR session, completing the bridge.

- createXrSwapchains() creates the render targets for the headset. It first queries the runtime for supported image formats (xrEnumerateSwapchainFormats) and recommended resolutions for each eye (xrEnumerateViewConfigurationViews). It then creates an XrSwapchain for each eye, which is a collection of images managed by the runtime. Finally, it enumerates these images (xrEnumerateSwapchainImages) to get their VkImage handles and creates corresponding VkImageViews for rendering. These are distinct from the desktop window's VkSwapchainKHR.
- createRenderPass() creates a render pass compatible with the format chosen for the XR swapchains (xrColorFormat).
- A dedicated, high-resolution depth buffer for VR is created in createXrDepthResources() using the dimensions recommended by the runtime. This is separate from the desktop mirror's depth buffer, which is sized to the window.
- createXrFramebuffers() iterates through the VkImageViews of each eye's XR swapchain and combines them with the shared xrDepthImageView to create a VkFramebuffer for every possible render target image.
- createGraphicsPipeline() creates a single graphics pipeline compatible with both XR and desktop mirror rendering.
- mainLoopXR() either calls xrWaitFrame, xrBeginFrame or xrEndFrame (through endXrFrame() method) based on XrFrameState. If it starts the frame (based on XrFrameState), drawXRFrame() is called. drawXRFrame() renders a frame for the VR headset by calling:
    - updateUniformBuffer(): camera projection matrices for both eyes and updates XrCompositionLayerProjectionView parameters for both eyes.
    - recordCommandBufferXR(): records command buffer for a specific eye's XR swapchain image.
- After the VR frame is submitted, the original drawFrame() function is called to perform a standard multi-pass render to the desktop window, serving as a spectator view.

By following this architecture, the Vulkan terrain renderer is successfully transformed from a desktop-only stereo application into a PC-tethered VR experience that is portable, effective, and aligned with modern XR development standards. The above steps, forming the PC-tethered VR architecture, can be seen in the call graph, Figure 5-6. Also, contributions of crucial methods are summarised in Figure 5-7.

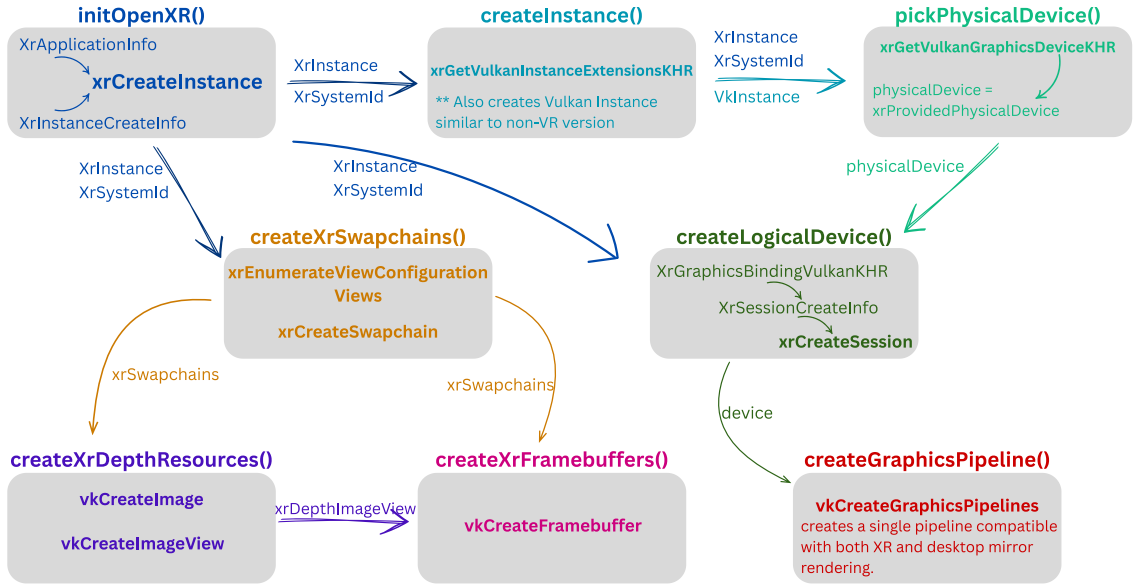


Figure 5-7 An overview of main OpenXR related methods in the implementation

5.11 Implementation of Data Collection

The time logs are recorded during the application's runtime, frame render times and frames per second (FPS). This data is then saved to CSV files for analysis.

1. Reserving Vectors (`frameTimeLog.reserve(150000), fpsLog.reserve(500)`) The `reserve()` method is called on `std::vector` objects within the `run()` method of the `TerrainRenderer` application. The purpose of `reserve()` is to pre-allocate memory for the vector to hold a specified number of elements. By using it, the vector avoids frequent reallocations as new elements are added, which can be computationally expensive and introduce latency.
2. The `mainLoop()` function is where the application's rendering and event processing occur. It contains the logic for calculating and storing frame-level and overall FPS data.
3. Inside the main loop, `currentTime` is captured at the beginning of each frame after `drawFrame()` completes.
4. `frameTimeSeconds` is calculated as the duration between `currentTime` and `renderLastTime`. This `frameTimeSeconds` represents the time taken to render the current frame.
5. If traversal count is less than or equal to `routeLimit`, this `frameTimeSeconds` value is then added to the `frameTimeLog` vector.
6. For FPS logging, the `mainLoop()` maintains counters frames and `lastTime` to track the number of frames rendered within a one-second interval.

Chapter 6 Results and Discussion

This chapter presents and analyses render time data collected from the five rendering implementations detailed in the previous chapter. All experiments were conducted on an NVIDIA GTX 1050 GPU to ensure that performance differences between rendering techniques were clearly observable. The analysis focuses on Frames Per Second (FPS) and application-level render times. Before initiating the main implementation stages, a preliminary benchmark was conducted to select the most suitable graphics API for the project's performance-critical goals. As shown in the initial findings table, Table 6-1, Vulkan consistently delivered superior performance compared to DirectX 11 on both the NVIDIA GTX 1050 and RTX 3070 GPUs, particularly as the geometric complexity of the mesh increased. Critically, on the GTX 1050, DirectX 11 failed with a `bad_alloc` error when attempting to render the 8-million-face mesh, highlighting a significant limitation in its ability to scale with large datasets. Given Vulkan's higher performance and, more importantly, its stability under high geometric load, it was selected as the foundational API for all subsequent development.

Table 6-1 Initial FPS results for moon terrain rendering

NVIDIA GTX 1050 num. faces	Vulkan (FPS)	DirectX 11 (FPS)
800k	~400	~200
2m	~400	~60
8m	~170	bad_alloc

NVIDIA RTX 3070 num. faces	Vulkan (FPS)	DirectX 11 (FPS)
800k	~8000	~3200
2m	~5000	~1200
8m	~1500	~400

6.1 Mesh Rendering with Varying Resolutions

The results in Figure 6-1 and Figure 6-2 demonstrate a clear and expected correlation between polygon count and performance. The low-resolution mesh (22k faces) maintained a very high and stable average FPS of approximately 400, with render times clustered in the fastest brackets. As the complexity increased to 1.164 million faces, the average FPS remained high and stable, though the render time distribution began to widen slightly, and FPS values slightly oscillated. A significant performance degradation was observed with the 6.354-million-face mesh, where the average FPS dropped to around 250 and exhibited greater instability. This confirms that while modern GPUs can handle moderately complex static meshes efficiently, performance becomes a limiting factor at very high polygon counts, underscoring the necessity for optimisation strategies such as Level of Detail (LOD).

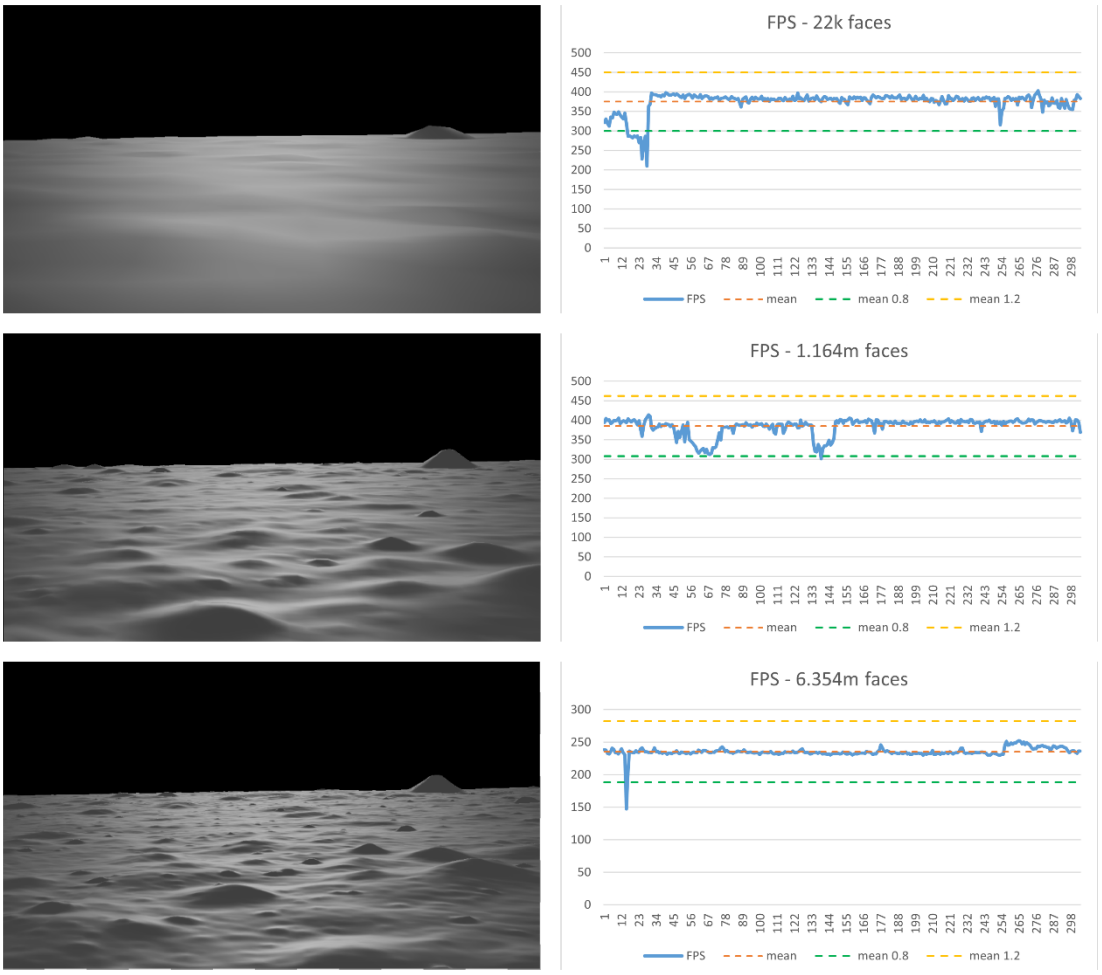


Figure 6-1 Visual Results and FPS graphs for rendering of varying mesh resolutions

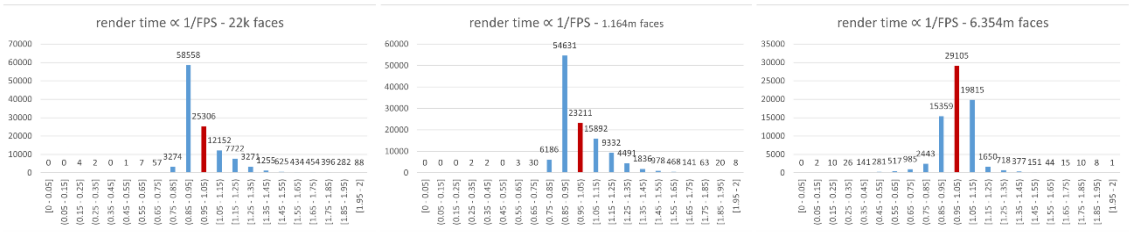


Figure 6-2 Render time histograms for varying mesh resolutions
The red bars indicates mean range (95%-105% of average render time value), so more value within (or close) that range means that less oscillation in render times, hence smoother performance.

6.2 LOD Rendering with Different Tessellation Levels

This experiment evaluated the effectiveness of the dynamic LOD system, which uses a low-resolution control grid and hardware tessellation to vary geometric detail based on camera distance. The results in Figure 6-3 and Figure 6-4 show that hardware tessellation is a highly efficient method for implementing LOD. With tessellation disabled (max. tessellation level is set to 1), performance was high, similar to rendering a simple mesh. Increasing the maximum tessellation level to 8 introduced a vast amount of dynamic detail to the scene with only a

negligible impact on the average FPS, which remained stable at approximately 350-400. Even at the highest quality setting (max level 16), the performance remained robust, with only a minor drop in average FPS. This outcome validates the strategy of offloading geometry generation to the GPU, which can dynamically create high-fidelity surfaces with minimal performance cost compared to rendering a dense, static mesh.

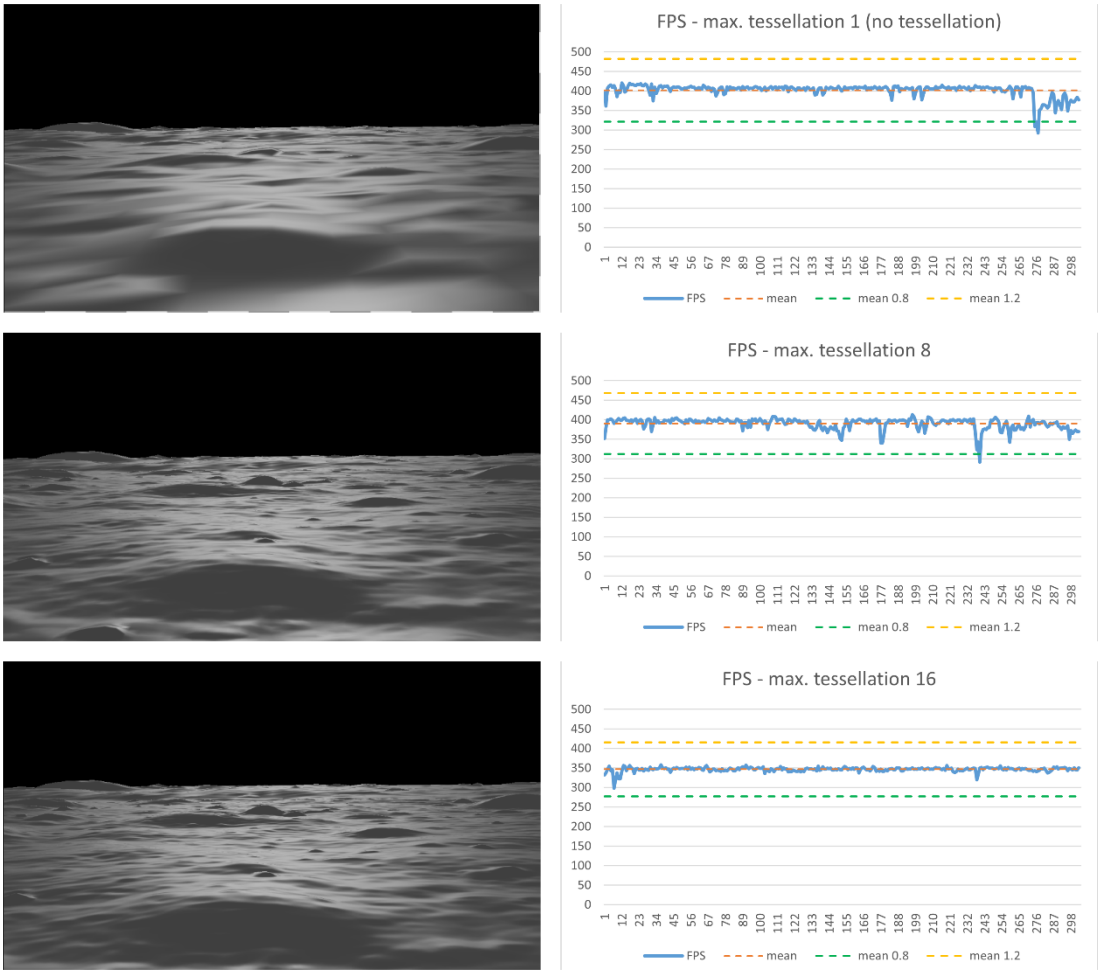


Figure 6-3 Visual Results and FPS graphs for rendering with different max. tessellation levels

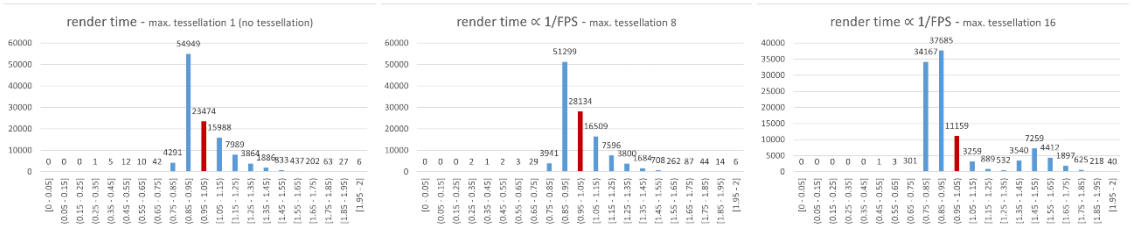


Figure 6-4 Render time histograms for rendering with different max. tessellation levels
The red bars indicates mean range (95%-105% of average render time value), so more value within (or closer) that range means that less oscillation in render times, hence smoother performance.

6.3 Stereo Rendering (Singlepass or Multipass)

It's a comparison between the two implemented stereo rendering techniques using the 1.164m face mesh to analyse the performance trade-offs between a hardware-level and a software-level approach. The results in Figure 6-5 demonstrate the performance superiority of the single-pass (multi-view) implementation. It consistently maintained a higher average FPS (around 300-350) and exhibited greater stability than the multi-pass version. The multi-pass renderer, which executes two draw calls per frame, averaged a lower FPS of around 250. This performance difference is attributed to the efficiency of the multi-view hardware feature, which allows the GPU to process geometry once and broadcast it to two view layers, significantly reducing CPU overhead compared to the software-driven approach of issuing two separate draw calls.

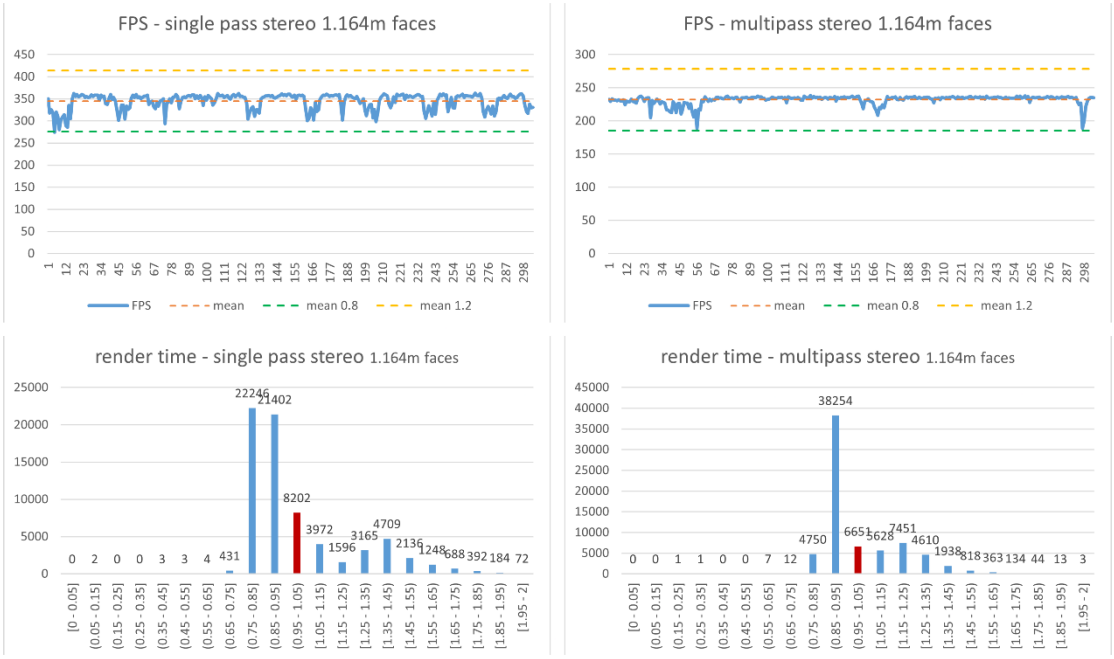


Figure 6-5 FPS graphs and render time histograms for stereo rendering
The red bars indicates mean range (95%-105% of average render time value), so more value within (or close) that range means that less oscillation in render times, hence smoother performance.

6.4 Stereo Rendering with and without Tessellation

This final implementation is the most computationally demanding, combining the dynamic geometry from the tessellation pipeline with the dual-draw approach of multi-pass stereo rendering. It serves as a comprehensive test of the Vulkan renderer's ability to handle a complex, high-workload scenario. The results in Figure 6-6 are particularly insightful, showing that the tessellated multi-pass stereo renderer performed similarly, yet slightly better than the static mesh multi-pass stereo renderer. Both configurations averaged a stable FPS of around 250. However, the render time distribution for the tessellated version is noticeably tighter and

slightly faster, indicating more consistent frame times. This demonstrates that the computational cost of running the tessellation shaders is effectively offset by the benefit of processing a much simpler base geometry on the CPU. The outcome shows that a tessellation-based LOD system is a powerful optimisation for mono rendering and a highly viable and efficient strategy for managing geometric complexity in more demanding stereo rendering workloads.

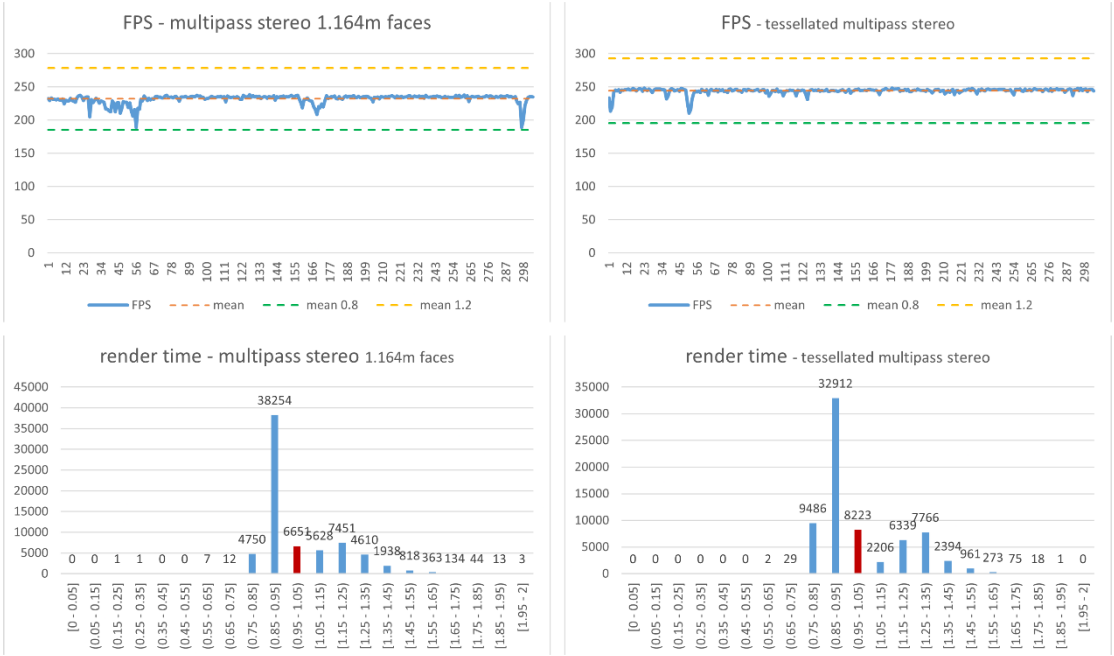


Figure 6-6 FPS graphs and render time histograms for multipass stereo rendering with and without tessellation

Chapter 7 Conclusion

This thesis investigates, implements and evaluates the rendering pipeline of a large-scale, high-fidelity lunar terrain model using the low-level Vulkan graphics API. The main goal was to move beyond the abstractions of high-level game engines to gain fine-grained control over the graphics pipeline, enabling a systematic analysis of advanced rendering techniques crucial for performance-critical applications like virtual reality. Through a series of iterative implementations, this study successfully navigated the complexities of data processing, dynamic LOD approach, and stereo rendering to deliver a performance analysis.

The project's objectives were methodically achieved. A foundational performance comparison between DirectX 11 and Vulkan conclusively demonstrated Vulkan's superior performance and, critically, its stability when handling high-polygon workloads, validating its selection for this study. The core achievement was the successful implementation of a dynamic LOD system using hardware tessellation. This technique proved highly effective, offloading geometry generation to the GPU to render visually complex terrain with a minimal performance overhead compared to rendering a dense, static mesh. Furthermore, the implementation and comparison of single-pass and multi-pass stereo rendering provided a clear insight into VR-like workloads. The results clearly favoured the hardware-accelerated single-pass approach, highlighting the significant efficiency gains offered by modern GPU features like multiview. The culmination of this work was a robust rendering pipeline capable of processing raw NASA LROC data into an optimised, real-time visualisation of the Apollo 11 landing site, which was also successfully integrated with the OpenXR SDK for a PC-tethered VR experience.

The path through this research yields significant understandings. The primary takeaway is the profound impact of low-level API control. While the learning curve for Vulkan is steep, its explicit nature was instrumental in implementing and fine-tuning features that are often obscured in higher-level engines. This direct control was essential for optimising the rendering pipeline and understanding the hardware-level trade-offs between different techniques. The study reaffirmed that for large-scale terrain, a dynamic LOD system is not merely an optimisation but a requirement. In particular, the efficiency of hardware tessellation demonstrated its power as a modern solution for generating on-the-fly geometric detail precisely where it is needed most. Reflecting on the project, several aspects for future work could build upon this foundation. The current implementation focused primarily on geometric complexity; a natural next step would be to incorporate an advanced texturing and material system, potentially exploring procedural generation or texture streaming techniques to further enhance the visual quality without compromising performance. While the tessellation-based

LOD was effective, future research could investigate hybrid approaches, such as combining tessellation with chunked LOD or modern mesh shaders, to potentially achieve even greater efficiency. The lighting model, currently a simple Phong implementation, could be expanded into a physically based rendering (PBR) system with dynamic shadows and atmospheric effects to create a more photorealistic and immersive lunar environment. Finally, while this project successfully targeted PC-tethered VR, a challenging and valuable future direction would be to adapt and optimise the rendering pipeline for the significant constraints of standalone VR hardware, a task that would demand a different suite of optimisation strategies.

In conclusion, this thesis successfully demonstrated the design and implementation of a high-performance rendering pipeline for large-scale planetary terrain. The findings validate the use of the Vulkan API for demanding scientific visualisation tasks and provide a quantitative analysis of modern rendering techniques. The research serves as a practical guide and a solid foundation for future development in the fields of real-time simulation, education, and immersive virtual reality experiences.

Legal, Ethical, Social and Professional Considerations

The study employs a technical development and experimentation methodology. A custom rendering application has been developed using C++ and Vulkan to display a high-fidelity 3D model of the lunar surface derived from publicly available NASA LROC data. Various real-time rendering variations have been implemented and compared, including tessellation-based LOD design and stereo rendering. Performance metrics such as frame rate, frame timing consistency, and resource utilisation have been collected through systematic testing. No human participants have been involved, and no personal data have been collected. The research focuses solely on software development and performance analysis, ensuring minimal ethical risks. This project has been evaluated for ethical concerns and poses minimal risk. No participants have been involved, as the study focuses on software development and performance evaluation. Therefore, issues of consent, privacy, or the welfare of subjects do not apply. No personal data has been collected for the project, nor involve any interactive user studies. All data used in the project are from public and reputable sources (e.g., NASA's LROC lunar dataset). This data aligns with its intended scientific and educational purpose. NASA data is in the public domain, and the data resource has been attributed to proper credit in the dissertation. No intellectual property conflicts are anticipated, as the project code benefits from original or use permissively licensed libraries (Vulkan, free to use; GDAL is open source and free to use). The source code is publicly available in public GitHub repositories; two separate repositories are delivered, the first one is for data processing, and the second one is for the lunar terrain rendering pipeline.

References

Al-Jundi, H. A. & Tanbour, E. Y. (2022) A framework for fidelity evaluation of immersive virtual reality systems. *Virtual reality : the journal of the Virtual Reality Society*, 26(3), 1103–1122. https://doi.org/10.1007/s10055-021-00618-y

Albayrak, D. (2025a) *Converting GeoTIFF to an .obj mesh and obtaining a heightmap.* https://github.com/DilaraAlbayrak/geotiff-to-mesh [Accessed 24 September].

Albayrak, D. (2025b) *Lunar Terrain Rendering using Vulkan.* https://github.com/DilaraAlbayrak/terrain-rendering-vulkan [Accessed 24 September].

Blondé, L., Doyen, D. & Borel, T. (2010) 3D stereo rendering challenges and techniques *2010 44th Annual Conference on Information Sciences and Systems (CISS)*.

Campbell, D. (2023) *Design of a High-Performance Computer Graphics Interface in a High-Level Programming Language*. MSc Master's Thesis. Wichita State University.

Cantlay, I. (2011) *Directx 11 terrain tessellation.* 11.NVIDIA Corporation.

Chaurasia, G., Nieuwoudt, A., Ichim, A.-E., Szeliski, R. & Sorkine-Hornung, A. (2020) Passthrough: Real-time Stereoscopic View Synthesis for Mobile Mixed Reality. *Proceedings of the ACM on computer graphics and interactive techniques*, 3(1), 1–17. https://doi.org/10.1145/3384540

Etienne, S. (2023) Large Scale Terrain Rendering in Call of Duty *SIGGRAPH 2023: The Premier Conference & Exhibition on Computer Graphics & Interactive Techniques*. Los Angeles, 6–10 August.

Hedberg, M. (2022) *Rendering of Large Scale Continuous Terrain Using Mesh Shading Pipeline*. MSc. MSc. Umeå University. https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1676474&dswid=2678 [Accessed June 2, 2025].

Hogan, J., Salo, A., Rzig, D. E., Hassan, F. & Maxim, B. (2022) Analyzing Performance Issues of Virtual Reality Applications. https://doi.org/10.48550/arxiv.2211.02013

Hoppe, H. (1998) Smooth view-dependent level-of-detail control and its application to terrain rendering 1998. 10.1109/VISUAL.1998.745282

Hosny, Y. S. S., Salem, M. A.-M. & Wahby, A. (2020) Performance optimization for standalone virtual reality headsets *2020 IEEE Graphics and Multimedia (GAME)*.

Jansen, M., Donkervliet, J., Trivedi, A. & Iosup, A. (2023) Can My WiFi Handle the Metaverse? A Performance Evaluation Of Meta's Flagship Virtual Reality Hardware New York, NY, USA, 2023. 10.1145/3578245.3585022

Khronos (2025a) *Overview.* https://docs.vulkan.org/tutorial/latest/01_Overview.html [Accessed 20 July 2025].

Khronos (2025b) *Shader modules.* https://docs.vulkan.org/tutorial/latest/03_Drawing_a_triangle/02_Graphics_pipeline_basics/01_Shader_modules.html [Accessed 20 July 2025].

Khronos, O. (2019) Khronos Highlights OpenXR 1.0 Specification. *Professional Services Close-Up*. https://go.exlibris.link/SN4Zh5ln

Krupip (2019) *Clarification on Vulkan multiview (stereoscopic 3D) features.* https://stackoverflow.com/a/57312456 [Accessed 16 Aug 2025].

Lai, Z., Hu, Y. C., Cui, Y., Sun, L. & Dai, N. (2017) Furion: Engineering High-Quality Immersive Virtual Reality on Today's Mobile Devices New York, NY, USA, 2017. 10.1145/3117811.3117815

Lauritsen, T. N., Steen Lund (2025) *Rendering Very Large, Very Detailed Terrains.* Danish Technical University, Informatics and Mathematical Modelling. https://www.terrain.dk/ [Accessed May 29, 2025].

Lee, H., Kim, J., Son, J.-Y., Kim, I., Noh, J., Yoon, Y.-J. & Yoon, M. (2022) Evaluation of eye response using a wearable display with automatic interpupillary distance adjustment. *Optics express*, 30(5), 8151. https://doi.org/10.1364/OE.453080

Lemenkova, P. & Debeir, O. (2023) GDAL and PROJ Libraries Integrated with GRASS GIS for Terrain Modelling of the Georeferenced Raster Image. *Technologies (Basel)*, 11(2), 46. https://doi.org/10.3390/technologies11020046

Lindner, C., Ortwein, A., Staar, K. & Rienow, A. (2021) Different Levels of Complexity for Integrating Textured Extra-terrestrial Elevation Data in Game Engines for Educational Augmented and Virtual Reality Applications. *Kartographische Nachrichten*, 71(4), 253–267. https://doi.org/10.1007/s42489-021-00090-3

LRO LROC Team, A. S. U., Mark Robinson, PhD (2020) *Apollo 11 Landing Site High-Resolution NAC Digital Terrain Model* [Map]. 2011-01-22T18:55:49 to 2011-01-22T20:49:11. (Arizona, USA) NAC_DTM_APOLLO11.LBL

Luebke, D. P. (2003) *Level of detail for 3D graphics*, 1st edition. Morgan Kaufmann.

Masood, Z., Jiangbin, Z., Ahmad, I. & Irfan, M. (2022a) High-performance adaptive texture streaming for planetary-scale high-mobility information visualization. *Journal of King Saud University. Computer and information sciences*, 34(10), 8336–8349. https://doi.org/10.1016/j.jksuci.2022.08.014

Masood, Z., Zheng, J., Ahmad, I., Irfan, M. & Saeed, A. (2022b) High-performance gpu based planetary-scale terrain visualization *Proceedings of the 2022 5th international conference on image and graphics processing*.

Microsoft (2023) *Graphics APIs in Windows.* https://learn.microsoft.com/en-us/windows/win32/direct3darticles/graphics-apis-in-windows-vista [Accessed July 31].

Müller, M. G., Durner, M., Gawel, A., Stürzl, W., Triebel, R. & Siegwart, R. (2021) A photorealistic terrain simulation pipeline for unstructured outdoor environments *2021 IEEE/RSJ international conference on intelligent robots and systems (IROS)*.

Overvoorde, A. (2025) *Vulkan tutorial. https://vulkan-tutorial.com/resources/vulkan_%20tutorial_en.pdf*.

Rendevski, N., Trajcevska, D., Dimovski, M., Veljanovski, K., Popov, A., Emini, N. & Veljanovski, D. (2022) PC VR vs standalone VR fully-immersive applications: History, technical aspects and performance *2022 57th international scientific conference on information, communication and energy systems and technologies (ICEST)*.

Ritter, N. & Ruth, M. (1997) The GeoTiff data interchange standard for raster geographic images. *International journal of remote sensing*, 18(7), 1637–1647. https://doi.org/10.1080/014311697218340

Shiraef, J. (2016) *An Exploratory Study of High Performance Graphics Application Programming Interfaces*. MSc Master's Thesis. The University of Tennessee at Chattanooga.

Sukaridhoto, S., Haz, A. L., Fajrianti, E. D. & Rizqi Putri Nourma, B. (2023) Comparative Study of 3D Assets Optimization of Virtual Reality Application on VR Standalone Device. *International Journal on Advanced Science, Engineering and Information Technology*, 13, 999–1008. https://doi.org/10.18517/ijaseit.13.3.18375

Suwandi, G. R. F., Khotimah, S. N., Haryanto, F. & Suprijadi, S. (2024) Effect of virtual reality video frame rate on visually induced motion sickness through electroencephalography measurement. *AIP conference proceedings*, 3210(1). https://doi.org/10.1063/5.0228169

Vulkan (2025) *What is SPIR-V.* https://docs.vulkan.org/guide/latest/what_is_spirv.html [Accessed 20 July 2025].

Wang, L., Shi, X. & Liu, Y. (2023) Foveated rendering: A state-of-the-art survey. *Computational Visual Media*, 9(2), 195–228. https://doi.org/10.1007/s41095-022-0306-4

Wendt, J. (2023) *A performance comparison between quadtree and tessellation LOD solutions for planetary terrain*. BSc Bachelor's Thesis. Blekinge Institute of Technology,.

Wong, F. (2016) Performance analysis and optimization for pc-based vr applications: From the cpu's perspective. *Intel Virtual Reality Documentation*, 16.

Yu, Y. & Du, Z. (2024) A GPU computation-based ray tracing engine with user-friendly and scalable rendering features and structures *Seventh International Conference on Computer Graphics and Virtuality (ICCGV 2024)*.

Процик, М. Ю. & Коротєєва, Т. О. (2025) Analysis the efficiency of programming tools for 3D rendering in video game development. *Naukovij vìsnik NLTU Ukraïni (Online)*, 35(3), 158–165. https://doi.org/10.36930/40350317